

Hessian-basierte Vorhersage von Quantisierungsschäden in CNNs

Schichtweise Sensitivitätsanalyse unter Power-of-Two Quantisierung

Dominic Jean-Luc Tarnowski

Betreuung: Prof. Thomas Manke, Prof. Tatiana Ermakova
School of Computing, Communication and Business, HTW Berlin

9. September 2026

Zusammenfassung

Dieser Bericht dokumentiert das Forschungsprojekt B. Im Zentrum steht die Frage, ob sich mithilfe Hessian-basierter Krümmungsmetriken vorhersagen lässt, welche Schichten eines Convolutional Neural Network (CNN) unter der Power-of-Two-Quantisierung (PoT) am stärksten geschädigt werden, wie Post-Training-Quantisierung (PTQ) und Quantization-Aware-Training (QAT) diese Krümmung dabei jeweils unterschiedlich prägen, und in welchem Ausmaß der Schaden überhaupt auf Gewichts- statt Aktivierungsquantisierung zurückgeht. Die PoT ist ein hardware-attraktives Quantisierungsschema, da sie Multiplikationen durch Bitshifts ersetzt. Gleichzeitig erzeugt sie jedoch ungleichmäßige Genauigkeitsverluste über die Schichten hinweg. Als etablierter Ansatz schlägt die HAWQ-V2-Formulierung hierfür das Produkt aus Hessian-Spur und Störungsgröße, $\text{Tr}(H) \cdot \|\Delta W\|^2$, als Sensitivitäts-Score vor. Um den Schaden jeder Schicht isoliert zu bestimmen, wird jeweils nur diese eine Schicht quantisiert und der daraus resultierende Anstieg des Validierungs-Loss gemessen. Auf dieser Grundlage werden drei Scores gegen den gemessenen Schaden getestet, die rohe Krümmung, die Störungsgröße und das HAWQ-V2-Produkt, und zwar über drei Architekturen (CNN, ResNet-18, ResNet-50) sowie zwei Datensätze (CIFAR10, ImageNet100). Die Ergebnisse zeigen, dass kein Score universell zuverlässig ist. Während die rohe Krümmung die am stärksten geschädigte Schicht am konsistentesten identifiziert, wird das HAWQ-V2-Produkt durch eine Anti-Korrelation zwischen Störungsgröße und tatsächlichem Schaden systematisch in die Irre geführt. Über die reine Prädiktor-Frage hinaus behandelt der Bericht die vollständige sechsstufige Analyse-Pipeline. Diese umfasst eine Random-Init-Kontrolle, eine gescheiterte KFAC-Zerlegung, eine Dekomposition in Gewichts- und Aktivierungsanteile sowie eine Deployment-Evaluation auf realer Hardware. Dabei zeigt sich, dass die PoT-Quantisierung auf einer NVIDIA A100 keinen Laufzeitvorteil bringt, da aktuelle INT8-Kernel für Faltungsschichten fehlen, während der CPU-Pfad mit echten INT8-Kernen (fbgemm) den intendierten Hardware-Vorteil mechanistisch realisiert. Ein zentrales methodisches Ergebnis ist zudem ein Rekonziliations-Ledger, das dokumentiert, wie stark publizierte Trace-Werte von Konfigurationsentscheidungen abhängen, die in der Literatur meist unausgesprochen bleiben. Die Ergebnisse der Kernfragestellung münden schließlich in ein Workshop-Paper, das bei AXIOM @ NeurIPS 2026 eingereicht wurde.

1 Einleitung

1.1 Motivation und Problemstellung

Neuronale Netze werden zunehmend auf ressourcenbeschränkter Hardware ausgeführt, etwa auf eingebetteten Systemen, mobilen Geräten und Edge-Beschleunigern. Auf solcher Hardware sind die Inferenzkosten eines vollpräzisen Modells oft nicht mehr tragbar. Die Quantisierung reduziert diese Kosten, indem die Gewichte und Aktivierungen eines Netzes von 32-Bit-Gleitkommazahlen (FP32) auf niedriger aufgelöste, diskrete Werte abgebildet werden, typischerweise auf 8 Bit oder weniger. Dies senkt sowohl den Speicherbedarf als auch, bei passender

Hardware-Unterstützung, die Rechenzeit pro Inferenz.

Eine besonders hardware-attraktive Variante ist die Power-of-Two-Quantisierung (PoT). Hierbei werden die Gewichte nicht auf ein gleichmäßiges, uniformes Gitter abgebildet, sondern auf exakte Zweierpotenzen. Der Vorteil liegt in der Arithmetik, da eine Multiplikation mit einer Zweierpotenz einem einfachen Bitshift entspricht. Dadurch können die sonst dominierenden Multiplizierer-Schaltkreise in der Hardware entfallen [17, 18]. Dieser Effizienzgewinn ist jedoch mit einem Nachteil verbunden. Da das PoT-Gitter logarithmisch und nicht linear ist, variiert der Quantisierungsfehler stark und unvorher-

sehbar mit den tatsächlichen Gewichtswerten einer Schicht. Dieses Verhalten unterscheidet sich grundlegend von der uniformen Quantisierung und hat, wie dieser Bericht zeigt, direkte Konsequenzen für die Anwendbarkeit klassischer Sensitivitätsmaße.

Um zu entscheiden, welche Schichten eines Netzes eine niedrigere Bit-Breite vertragen und welche nicht, hat sich die Verwendung von Krümmungsinformation aus der zweiten Ableitung des Verlustes etabliert, also der Hessian-Matrix. Der einflussreichste Vertreter dieses Ansatzes ist die HAWQ-Familie [3, 2]. HAWQ rangiert die Schichten nach ihrem größten Hessian-Eigenwert, während HAWQ-V2 dies zu einem Produkt aus Hessian-Spur und quadrierter Störungsgröße, $\text{Tr}(H) \cdot \|\Delta W\|^2$, verfeinert, motiviert durch eine Taylor-Entwicklung des Verlustes zweiter Ordnung. Beide Arbeiten nutzen diese Formulierung, um eine Bit-Zuteilung über das gesamte Netz zu optimieren. Validiert wird dabei die End-to-End-Genauigkeit *nach* der Zuteilung und nicht die Vorhersagegüte des Scores auf Ebene einzelner Schichten.

Genau hier setzt dieses Projekt an. Die leitende Frage lautet, ob die HAWQ-V2-Formulierung tatsächlich vorhersagt, welche einzelne Schicht unter der PoT-Quantisierung am meisten leidet. Diese Frage ist nicht rein akademisch. Nutzen Praktiker Hessian-basierte Scores, um zu entscheiden, welche Schichten für gemischte Präzision reserviert werden, so verlassen sie sich implizit darauf, dass der Score mit dem tatsächlichen Schaden korreliert. Diese Annahme wurde nach bisherigem Kenntnisstand vor diesem Projekt für die PoT-Quantisierung an Vision-CNNs nie direkt gegen gemessenen Schaden getestet.

1.2 Forschungsfragen

Das Projekt verfolgt drei Forschungsfragen:

- F1** Wie prägen PTQ und QAT die schichtweise Verlust-Krümmung (gemessen über die Hessian-Spur) von CNNs unter PoT-Quantisierung jeweils unterschiedlich?¹
- F2** Sagt die schichtweise Verlust-Krümmung (Hessian-Spur) vorher, welche Schichten tatsächlich den größten PoT-Gewichts-Quantisierungsschaden erleiden, und wie hängen diese beiden Auswahlen (vorhergesagt

vs. tatsächlich) von Architektur und Datensatz ab?²

- F3** In welchem Ausmaß ist der PoT-induzierte Genauigkeitsverlust auf Gewichts- gegenüber Aktivierungsquantisierung zurückführbar, und für welches Regime ist demzufolge die Hessian-basierte Sensitivitätsanalyse anwendbar?³

F2 ist die zentrale Fragestellung und der Kern des daraus entstandenen Workshop-Papers (§7). F1 und F3 sind breiter angelegt und werden in diesem Bericht vollständig behandelt, obwohl sie im Papier aus Platzgründen nur gestreift oder ausgeklammert werden.

1.3 Beiträge dieses Projekts

- Ein empirischer, direkter Test der HAWQ-V2-Formulierung unter PoT-Quantisierung. Der Schaden jeder Schicht wird einzeln isoliert (§4) und es werden drei Sensitivitäts-Scores gegen den tatsächlich gemessenen Loss-Schaden verglichen, nicht gegen die End-to-End-Genauigkeit nach einer Bit-Zuteilung wie in der ursprünglichen HAWQ-Literatur.
- Eine Dokumentation der Konfigurations-Sensitivität von Hessian-Trace-Messungen. Ein Rekonziliations-Ledger (§5) gleicht historische, unterschiedlich konfigurierte Trace-Werte für dieselbe Schicht mit einer neu festgelegten kanonischen Konfiguration ab. Dabei zeigt sich, dass Unterschiede von zwei Größenordnungen auftreten können, ohne dass eine einzelne Konfigurationsentscheidung sie erklärt.
- Eine Deployment-Evaluation auf realer Hardware (NVIDIA A100, x86-CPU mit AVX-512 VNNI), die zeigt, dass der theoretische Bitshift-Vorteil von PoT nicht automatisch in einen gemessenen Laufzeitvorteil übersetzt.
- Eine sechsheufige Analyse-Pipeline, die über die im Workshop-Paper berichteten Ergebnisse hinausgeht und zusätzlich eine Random-Init-Kontrolle, die KFAC-Faktorisierung, die Aktivierungs-Analyse sowie die QAT-Ergebnisse umfasst.

²Im Original: *Does layer-wise loss curvature (Hessian trace) predict which layers actually incur PoT weight-quantization damage, and how do these two selections depend on architecture and dataset?*

³Im Original: *To what extent is PoT-induced accuracy loss attributable to weight vs. activation quantization, and consequently, to which regime does Hessian-based sensitivity analysis apply?*

¹Im Original: *How do PTQ and QAT differently reshape the layer-wise loss curvature (measured via Hessian trace) of CNNs under PoT quantization?*

- Die Einreichung eines Workshop-Papers bei AXIOM @ NeurIPS 2026, das die Kernergebnisse zu F2 in kompakter Form für die Fachcommunity aufbereitet.

1.4 Aufbau des Berichts

§2 führt die Grundlagen ein, die für das Verständnis der Methodik nötig sind: Quantisierung, die Hessian-Matrix, HAWQ/HAWQ-V2, Hessian-Spektraltheorie und Aktivierungsquantisierung. §3 positioniert das Projekt relativ zur Literatur. §4 beschreibt das experimentelle Setup und alle sechs Analysephasen im Detail. §5 präsentiert die Ergebnisse jeder Phase mit den zugehörigen Tabellen und Abbildungen. §6 interpretiert die Befunde, benennt Limitationen und skizziert Ausblick. §7 fasst zusammen und verweist auf das eingereichte Workshop-Paper.

2 Grundlagen

Dieser Abschnitt führt alle Konzepte ein, die für das Verständnis der Methodik (§4) und der Ergebnisse (§5) notwendig sind. Er richtet sich an Leser mit ML-Hintergrund, die nicht notwendigerweise mit Quantisierung oder Krümmungsanalyse vertraut sind.

2.1 Quantisierung neuronaler Netze

Grundidee. Ein trainiertes neuronales Netz speichert seine Gewichte und berechnet seine Aktivierungen typischerweise als 32-Bit-Gleitkommazahlen (FP32). Quantisierung bildet diese kontinuierlichen Werte auf eine kleinere, diskrete Menge von Repräsentanten ab, üblicherweise mit deutlich weniger Bits pro Wert. Das reduziert den Speicherbedarf direkt proportional zur Bit-Breite und kann, wenn die Zielhardware quantisierte Arithmetik-Einheiten besitzt, auch die Rechenzeit senken.

PTQ vs. QAT. Es gibt zwei grundsätzliche Strategien, ein Netz zu quantisieren. Bei der Post-Training-Quantisierung (PTQ) wird ein bereits fertig trainiertes FP32-Modell nachträglich quantisiert, ohne weiteres Training. PTQ ist schnell und benötigt keine Trainingsdaten in großem Umfang, produziert aber typischerweise größere Genauigkeitsverluste, da das Modell nie gelernt hat, mit dem Quantisierungsfehler umzugehen. Beim Quantization-Aware-Training (QAT) wird der Quantisierungseffekt bereits während des Trainings simuliert, sodass das Modell seine Gewichte an die Quantisierung anpassen kann. QAT erzielt in der Regel höhere Genauigkeit, benötigt aber zusätzliches Training.

Fake-Quantization und Straight-Through-Estimator. Sowohl bei der PTQ-Kalibrierung als auch bei QAT wird der Quantisierungseffekt üblicherweise durch eine Fake-Quantization simuliert. Dabei wird ein Wert quantisiert und sofort wieder in Gleitkomma zurückkonvertiert (oft als „QDQ“ bezeichnet), sodass der Rest der Berechnung weiterhin in FP32 abläuft, den Quantisierungsfehler aber dennoch „sieht“. Das eigentliche Problem liegt in der Rundungsfunktion in der Mitte dieser Operation, deren Ableitung fast überall null ist und die den Gradientenfluss beim Backpropagation-Training blockieren würde. Der Straight-Through-Estimator (STE, [1]) löst dies, indem er im Vorwärtsthroughlauf die tatsächliche, nicht-differenzierbare Quantisierung anwendet, im Rückwärtsthroughlauf jedoch so verfährt, als wäre die Quantisierungsfunktion die Identität. Der Gradient wird also unverändert durchgereicht. Diese Relaxierung ist für dieses Projekt relevant, da sie auch bestimmt, welche Krümmung auf quantisierten Modellen tatsächlich gemessen wird (§2.2).

Power-of-Two-Quantisierung. Während die uniforme Quantisierung die Werte auf ein gleichmäßig verteiltes Gitter abbildet (etwa 256 äquidistante Stufen bei 8 Bit), rundet die PoT-Quantisierung jeden Gewichtswert auf die nächste Zweierpotenz innerhalb eines Exponentenbereichs [17]. Der entscheidende Vorteil ist arithmetischer Natur, da eine Multiplikation $x \cdot 2^k$ auf Digitalhardware einem Bitshift um k Stellen entspricht. Dieser ist deutlich billiger als eine allgemeine Multiplikation und ohne dedizierten Multiplizierer realisierbar [18]. Der Nachteil ist ein nicht-uniformes Fehlerverhalten. Da das PoT-Gitter logarithmisch ist, ist die Gitterauflösung nahe null sehr fein und wird zu größeren Beträgen hin exponentiell gröber. Ob eine Schicht durch die PoT-Rundung stark oder kaum gestört wird, hängt somit weniger von einer festen Bit-Breite ab, wie bei der uniformen Quantisierung, als davon, wie gut die spezifische Verteilung der Gewichtswerte einer Schicht zufällig mit dem PoT-Gitter übereinstimmt. Dieser Punkt ist zentral für das Kernergebnis dieses Projekts (§6).

2.2 Die Hessian-Matrix und Krümmungsanalyse

Was die Hessian-Matrix ist. Für eine Verlustfunktion $L(W)$, die von den Gewichten W eines Netzes abhängt, ist die Hessian-Matrix $H = \nabla_W^2 L$ die Matrix der zweiten partiellen Ableitungen des Loss nach jedem Gewichtspaar. Sie beschreibt, wie stark sich die Steigung des Loss ändert, wenn man sich

entlang einer Richtung im Gewichtsraum bewegt, anschaulich gesprochen also, wie stark die Verlustlandschaft an der aktuellen Position gekrümmt ist.

Warum sie für Quantisierung relevant ist. Quantisierung stört die Gewichte um einen kleinen Betrag $\Delta W = W - Q(W)$, wobei $Q(\cdot)$ die Quantisierungsfunktion ist. Eine Taylor-Entwicklung des Loss um den unquantisierten Punkt zweiter Ordnung ergibt

$$\Delta L \approx \nabla L^\top \Delta W + \frac{1}{2} \Delta W^\top H \Delta W, \quad (1)$$

wobei der lineare Term bei einem konvergierten Modell (Gradient nahe null) vernachlässigbar ist, sodass der quadratische, Hessian-gewichtete Term den Loss-Anstieg dominiert. Diese Beziehung ist die theoretische Grundlage dafür, die Krümmung als Proxy für die Quantisierungsempfindlichkeit zu verwenden. Laut Gleichung (1) sollte eine Schicht mit hoher Krümmung empfindlicher auf eine Störung gleicher Größe reagieren als eine Schicht mit niedriger Krümmung.

Hutchinson-Schätzer für die Spur. Für moderne Netze mit Millionen Parametern ist die vollständige Hessian-Matrix weder speicherbar noch berechenbar. Praktikabel ist jedoch die Spur $\text{Tr}(H) = \sum_i H_{ii}$, die grob gesprochen die aufsummierte Krümmung über alle Gewichtsrichtungen angibt. Der Hutchinson-Schätzer approximiert die Spur stochastisch. Für zufällige Vektoren v mit $\mathbb{E}[vv^\top] = I$ (üblicherweise Rademacher-Vektoren, deren Komponenten gleichverteilt ± 1 sind) gilt $\text{Tr}(H) = \mathbb{E}[v^\top H v]$. Das Produkt Hv , ein Hessian-Vektor-Produkt, lässt sich effizient per automatischer Differentiation berechnen, ohne H jemals explizit zu bilden. Mehrere solcher Zufallsproben werden gemittelt, um die Schätzung zu stabilisieren.

PyHessian. Yao u. a. [22] implementieren diesen Hutchinson-Ansatz (inklusive Top-Eigenwert-Schätzung über Power-Iteration) als wiederverwendbares Werkzeug für PyTorch-Modelle und etablieren ihn als De-facto-Standard für Krümmungsanalysen in der Quantisierungsliteratur. Dieses Projekt verwendet PyHessian als Grundlage der eigenen Trace-Messungen (§4).

2.3 HAWQ und HAWQ-V2

Dong u. a. [3] führen HAWQ ein. Dabei werden die Schichten nach ihrem größten Hessian-Eigenwert rangiert, und Schichten mit größerem Eigenwert erhalten mehr Bits in einem gemischt-präzisen Quantisierungsschema. Dong u. a. [2] verfeinern dies zu HAWQ-V2. Statt nur des Eigenwerts wird nun ein

Sensitivitäts-Score als Produkt aus Hessian-Spur und quadrierter Störungsgröße gebildet, $\text{Tr}(H_l) \cdot \|\Delta W_l\|^2$ für Schicht l , direkt motiviert durch Gleichung (1), wobei $\text{Tr}(H_l)$ als Stellvertreter für die Krümmung dient.

Entscheidend ist, wofür diese Arbeiten ihre Formulierung nutzen und wofür nicht. Beide verwenden den Score, um eine globale Bit-Zuteilung über alle Schichten zu optimieren, und validieren den Erfolg dieser Zuteilung anhand der End-to-End-Genauigkeit des gesamten Netzes nach der Quantisierung. Keine der beiden Arbeiten testet direkt, ob der Score auf Ebene einzelner Schichten tatsächlich mit dem dort auftretenden Schaden korreliert. Ein Netz kann insgesamt eine gute Endgenauigkeit erreichen, obwohl der Score einzelne Schichten falsch einordnet, solange sich die Fehler kompensieren oder die insgesamt zugeteilten Bits ausreichen. Genau diese Lücke, die per-Schicht-Vorhersagegüte des HAWQ-V2-Scores direkt gegen gemessenen Schaden zu testen, adressiert dieses Projekt.

2.4 Hessian-Spektrum und Random-Matrix-Theorie

Über die Spur hinaus lässt sich das Spektrum, also die Menge der Eigenwerte, der Hessian-Matrix untersuchen. Sagun, Bottou und LeCun [19] und Sagun u. a. [20] zeigen empirisch, dass das Hessian-Spektrum tiefer neuronaler Netze typischerweise aus zwei Teilen besteht, einem Bulk nahe null, der die meisten Eigenwerte enthält, und wenigen großen Ausreißer-Eigenwerten, die mit der Anzahl der Klassen des Klassifikationsproblems zusammenhängen. Diese Bulk-Ausreißer-Trennung motiviert die Vorstellung, dass ein Netz effektiv nur entlang weniger Richtungen scharf gekrümmt ist.

Karakida, Akaho und Amari [9] liefern hierzu eine theoretische Grundlage aus der Random-Matrix-Theorie (RMT). Für zufällig initialisierte, untrainierte Netze lässt sich die Fisher-Information, die eng mit der Hessian-Matrix nahe einem Minimum verwandt ist, im Mittelfeld-Limit vorhersagen, inklusive einer charakteristischen Tiefenabhängigkeit. Diese Vorhersage ist der theoretische Bezugspunkt für die Random-Init-Kontrolle dieses Projekts (Phase 2, §5.3). Sie erlaubt zu unterscheiden, ob eine beobachtete Krümmungsspitze in einer bestimmten Schicht bereits bei Initialisierung vorhanden ist („architektonisch“, durch die Netzstruktur selbst erklärbar) oder erst durch Training entsteht („gelernt“).

Eine weitere Zerlegungsidee ist die Kroneckerfaktorierte Näherung der Krümmung (KFAC, Mar-

tens und Grosse [16]) sowie ihre Erweiterung auf Faltungsschichten (KFC, Grosse und Martens [5]). Die Grundidee besteht darin, die Fisher-Information einer einzelnen Schicht näherungsweise als Kronecker-Produkt zweier kleinerer Matrizen zu faktorisieren, $F \approx A \otimes G$, wobei A die Kovarianz der Schicht-Eingaben und G die Kovarianz der Verlust-Gradienten bezüglich der Schicht-Ausgaben erfasst. Gelingt diese Faktorisierung, so ließe sich die Spur einer Schicht in einen Eingabe-getriebenen und einen Gradienten-getriebenen Anteil zerlegen, was eine potenziell aufschlussreiche Diagnose dafür wäre, warum eine Schicht hohe Krümmung aufweist. Sowohl die RMT-Vorhersage als auch die KFAC-Zerlegung waren für dieses Projekt vielversprechende Werkzeuge, um die architektonischen Ursachen beobachteter Krümmungsspitzen zu erklären (Phase 6, §5.8). Wie in §5 gezeigt wird, führte insbesondere die KFAC-Zerlegung im PoT-Kontext jedoch nicht zum Ziel, da die vorhergesagte und die gemessene Spur um mehrere Größenordnungen voneinander abweichen.

2.5 Aktivierungsquantisierung und Range-Ausreißer

Alle bisherigen Konzepte betreffen die Quantisierung von Gewichten. Ein separates, in der Literatur gut dokumentiertes Problem ist die Quantisierung von Aktivierungen, also der Zwischenausgaben einer Schicht während der Inferenz. Aktivierungen folgen einem anderen Schadensmechanismus als Gewichte. Statt gleichmäßig verteilter Fehler treten oft wenige, aber extreme Ausreißer-Kanäle auf, deren Wertebereich um Größenordnungen über dem Rest liegt. Ein einzelner Skalierungsfaktor pro Tensor muss diesen gesamten Bereich abdecken, was die effektive Auflösung für alle übrigen, unauffälligen Kanäle drastisch reduziert. Xiao u. a. [21] (SmoothQuant) und Lin u. a. [15] (AWQ) zeigen dies für große Sprachmodelle und schlagen vor, die Schwierigkeit vor der Quantisierung von den Aktivierungen auf die leichter quantisierbaren Gewichte zu verschieben. Li u. a. [14] (BRECQ) verfolgt einen alternativen, blockweisen Rekonstruktionsansatz für PTQ, der Aktivierungs- und Gewichtsquantisierung gemeinsam über kleine Netzblöcke optimiert.

Für dieses Projekt ist relevant, dass Aktivierungsausreißer kanalweise auftreten und primär von der Eingabeverteilung bestimmt werden, während die Hessian-Krümmung eine Eigenschaft der Gewichte relativ zum Loss ist. Es gibt keinen a-priori-Grund, warum ein Hessian-basierter Score, der für Gewichts-schaden entwickelt wurde, auch Aktivierungsschaden

vorhersagen sollte. Diesen Punkt greift die Dekomposition in Gewichts- und Aktivierungsanteile (Phase 1, §4.5) empirisch auf.

3 Verwandte Arbeiten

Die HAWQ-Familie und ihre Grenzen. HAWQ [3] und HAWQ-V2 [2] etablieren die Hessian-Krümmung als Signal für die Bit-Zuteilung in gemischt-präzisen Quantisierungsschemata (§2.3). Beide Arbeiten validieren ihren Ansatz End-to-End, das heißt, nach der Bit-Zuteilung wird die Genauigkeit des gesamten Netzes gemessen und nicht die Vorhersagegüte des Scores für einzelne Schichten. Diese Validierungsstrategie ist für den ursprünglichen Zweck, eine gute globale Zuteilung zu finden, angemessen. Sie lässt jedoch offen, ob der Score tatsächlich lokal korrekt ist, denn ein Netz kann trotz einer schlecht kalibrierten Einzelschicht-Rangfolge insgesamt gut abschneiden, wenn genügend Gesamt-Bit-Budget vorhanden ist oder sich die Fehler zwischen den Schichten teilweise kompensieren.

Hill (2026): der erste direkte Test im LLM-Setting. Hill [7] testen als erste Arbeit direkt, ob die HAWQ-V2-Metrik $\text{Tr}(H) \cdot \|\Delta W\|^2$ mit tatsächlich gemessenem Quantisierungsschaden korreliert und nicht nur mit der End-to-End-Genauigkeit nach der Bit-Zuteilung. Das Setting dieser Arbeit ist jedoch ein großes Sprachmodell (LLM) unter uniformer Quantisierung und kein Vision-CNN unter PoT. Da PoT, wie in §2.1 erläutert, ein grundlegend anderes, nicht-uniformes Fehlerverhalten aufweist, ist unklar, ob sich die dortigen Befunde auf PoT-CNNs übertragen lassen. Diese Übertragung, und ihr Scheitern in einem entscheidenden Punkt (§6), ist ein zentraler Beitrag dieses Projekts.

PoT-Quantisierungsliteratur. Miyashita, Lee und Murmann [17] führen PoT als hardwarefreundliche logarithmische Gewichtsrepräsentation ein, Lee u. a. [12] zeigen den zugehörigen Energie-Vorteil logarithmischer Berechnung für CNNs, und Przewłocka-Rus u. a. [18] motivieren PoT zusätzlich für Hardware mit niedriger Bit-Breite. Neuere Arbeiten wie Kim u. a. [11] und Li u. a. [13] verfeinern die PoT-Gitterformulierung selbst, um bei sehr niedriger Bit-Breite höhere End-to-End-Genauigkeit zu erreichen, und Elgendedy [4] überträgt PoT auf Sprachmodell-Gewichte. Alle diese Arbeiten fokussieren, wie Miyashita, Lee und Murmann [17] und Przewłocka-Rus u. a. [18], auf die Machbarkeit, End-to-End-Genauigkeit und den Hardware-Vorteil von PoT und nicht auf eine schichtweise Sensitivitätsanalyse. In der PoT-Praxis, auch in diesem Projekt

(§4), ist es zudem üblich, die erste Faltungsschicht (**conv1**) entweder von der Quantisierung auszunehmen oder ihr besondere Aufmerksamkeit zu widmen, da sie mit ihren wenigen Eingabekanälen (meist drei Farbkanäle) und dem dadurch kleinen Fan-in empirisch besonders empfindlich reagiert. Dieses Projekt liefert mit der Random-Init-Kontrolle (Phase 2) und der Fusion-Analyse (Phase 3.5) erstmals eine systematische Erklärung dafür, warum **conv1** in dieser Hinsicht eine Sonderrolle einnimmt.

Aktivierungsquantisierung. SmoothQuant [21] und AWQ [15] zeigen, dass Aktivierungsschaden primär durch kanalweise Ausreißer getrieben wird, also durch einen von der Gewichtskrümmung unabhängigen Mechanismus (§2.5). BRECQ [14] adressiert PTQ durch eine blockweise Rekonstruktion, die Gewichts- und Aktivierungsfehler gemeinsam optimiert, ohne explizit die Hessian-Krümmung als Sensitivitätssignal zu nutzen. Diese Arbeiten motivieren, warum dieses Projekt Gewichts- und Aktivierungsschaden explizit trennt (Phase 1, §4.5), statt beide unter einem gemeinsamen Score zu behandeln.

Die Lücke. Zusammenfassend lässt sich festhalten, dass vor diesem Projekt kein publizierter empirischer Test existiert, der die HAWQ-V2-Formulierung direkt gegen gemessenen, per-Schicht-isolierten Gewichtsschaden für Vision-CNNs unter PoT-Quantisierung prüft. Dieses Projekt schließt diese Lücke und liefert zusätzlich die methodische Infrastruktur (kanonische Trace-Konfiguration, Rekonziliations-Ledger), die für eine reproduzierbare Beantwortung dieser Frage notwendig ist.

4 Methodik

4.1 Experimentelles Setup

Architekturen. Untersucht werden drei Convolutional-Architekturen unterschiedlicher Tiefe: ein kompaktes, projektspezifisches **cnn** mit sechs quantisierbaren Schichten (vier Faltungs-, zwei vollverbundene Schichten), **ResNet-18** [6] mit 21 quantisierbaren Schichten und **ResNet-50** [6] mit 54 quantisierbaren Schichten. Beide ResNet-Varianten werden *ohne* vortrainierte Gewichte instanziiert (`resnet18_no_weights`, `resnet50_no_weights`) und vollständig von Grund auf trainiert, um Confounds durch ImageNet-1k-Vortraining auszuschließen. Die drei Architekturen spannen eine Größenordnung an Schichtzahl auf ($n = 6, 21, 54$), was für die Normalisierung der Top- k -Auswertung in §4.4.1 relevant wird.

Datensätze. Verwendet werden CIFAR10 (32×32 Pixel, 10 Klassen) und ImageNet100 (224×224 Pixel, ein 100-Klassen-Subset von ImageNet). Diese Wahl deckt sowohl kleine als auch großformatige Eingaben ab und erlaubt es, Effekte, die von der Eingabeauflösung abhängen (etwa die Speicher- und Rechenkosten der Hessian-Schätzung, §4.2), von architekturbedingten Effekten zu trennen.

Training. Jede der sechs Architektur $\times$ Datensatz-Kombinationen wird mit einem Trainings-Seed trainiert (kein Multi-Seed-Training; siehe Limitationen, §6.6). Für jede Kombination werden drei Modellzustände erzeugt: das FP32-Basismodell, eine PTQ-Variante (FP32-Modell nachträglich PoT-quantisiert) und eine QAT-Variante (mit simulierter Quantisierung von Grund auf oder fortgesetzt trainiert). Trainings-Metadaten (Trainings-/Validierungsgenauigkeit, Wall-Time) sind in Tabelle 1 zusammengefasst.

PoT-Quantisierung. Die PoT-Implementierung ist exponentenbereich-basiert und nicht über eine feste Bit-Breite parametrisiert. Jeder Gewichtswert wird pro Tensor auf die nächste Zweierpotenz innerhalb eines festgelegten Exponentenbereichs gerundet. Dies folgt der in §2.1 beschriebenen PoT-Definition und ist konsistent über PTQ und QAT hinweg.

4.2 Kanonische Trace-Konfiguration

Ein zentraler methodischer Befund dieses Projekts (im Detail in §5.4) ist, dass gemessene Hessian-Trace-Werte empfindlich von Konfigurationsentscheidungen abhängen, die die Literatur meist implizit lässt: von der Reduktionsart des Loss (Mittelwert vs. Summe über ein Batch), der Basis des Modells (fusioniert vs. unfusioniert bezüglich Batch-Normalisierung), dem verwendeten Datensplit, der Anzahl der Hutchinson-Proben und dem Zufalls-Seed der Proben-Vektoren. Um reproduzierbare, vergleichbare Trace-Werte über die Trainingsstufen (FP32/PTQ/QAT) und Schichten hinweg zu erhalten, wird vor der Hauptanalyse eine kanonische Konfiguration festgelegt, dokumentiert in `trace_config.json`:

- **Reduktion:** mittelwertreduzierte Kreuzentropie, nicht Summe.
- **Basis:** fusionierte Batch-Normalisierungs-Basis [8] als kanonisch für stufenübergreifende Vergleiche, da PTQ- und QAT-Checkpoints ausschließlich in fusionierter Form existieren. FP32-Modelle werden zusätzlich in unfusionierter

Form vermessen, sodass der Fusionseffekt selbst sichtbar bleibt (Phase 3.5, §5.5).

- **Datensplit:** feste Validierungsbilder (kein zufälliges Neu-Sampling zwischen Messungen).
- **Proben-Seeds:** drei feste Seeds (42, 43, 44), deren Ergebnisse zu Mittelwert $\pm$ Standardabweichung pro Schicht aggregiert werden.
- **Determinismus:** deterministische Algorithmen, `eval()`-Modus (kein Dropout, feste BN-Statistiken).

Proben-Budget. Das Proben-Budget ist innerhalb eines Datensatzes fest, unterscheidet sich aber zwischen den Datensätzen: 80 Bilder mit 100 Hutchinson-Iterationen für CIFAR10 gegenüber 24 Bildern mit 30 Iterationen für ImageNet100. Der Grund ist rein rechnerischer Natur, da ein Hessian-Vektor-Produkt bei 224×224 -Eingaben (ImageNet100) um ein Vielfaches teurer ist als bei 32×32 (CIFAR10), sowohl in der Rechenzeit als auch im Speicherbedarf. Dieses ungleiche Budget ist eine dokumentierte Limitation (§6.6), denn Messrauschen könnte zur schwächeren Korrelation auf CIFAR10 beitragen, unabhängig von einem echten Datensatzeffekt.

STE-relaxierte Hessian. Da PTQ- und QAT-Gewichte über den Straight-Through-Estimator (§2.1) quantisiert wurden, ist die auf diesen Modellen gemessene Hessian-Matrix genau genommen die STE-relaxierte Krümmung des Loss, also die Krümmung, die sich ergibt, wenn die nicht-differenzierbare Quantisierungsfunktion im Rückwärtsdurchlauf durch die Identität ersetzt wird, und nicht die klassische Hessian eines unquantisierten Netzes. Diese Unterscheidung ist für die Interpretation der PTQ- und QAT-Trace-Werte in §5 wichtig.

4.3 Gewichts-Schaden-Isolation

Um den Schaden einer einzelnen Schicht l zu isolieren, werden ausschließlich die Gewichte dieser einen Schicht per PoT quantisiert, während alle übrigen Gewichte sowie alle Aktivierungen im gesamten Netz auf FP32 verbleiben. Implementiert wird dies, indem die Fake-Quantization-Module für Gewicht (`weight_fake_quant`) und Aktivierung (`act_fake_quant`) aller anderen Schichten durch `nn.Identity()` ersetzt werden. Nur die Zielschicht behält ihre echte Quantisierungslogik.

Loss-Schaden-Definition. Für jede Schicht l wird der Loss-Schaden definiert als

$$\text{loss_damage}(l) = \text{loss}_{\text{iso}}(l) - \text{loss}_{\text{fp32}}, \quad (2)$$

wobei $\text{loss}_{\text{iso}}(l)$ der mittlere Validierungs-Kreuzentropie-Loss des Modells mit isoliert quantisierter Schicht l ist (identische Loss-Definition wie die Trace-Schätzung in §4.2) und $\text{loss}_{\text{fp32}}$ der Loss des unveränderten FP32-Modells. Ein positiver Wert bedeutet, dass der Loss durch die Quantisierung gestiegen, also Schaden entstanden ist. Als primäres Ranking-Ziel wird $\text{abs_loss_damage}(l) = |\text{loss_damage}(l)|$ verwendet, da unter QAT auch negative Werte auftreten (die isolierte Quantisierung einer Schicht kann den Loss gegenüber der FP32-Baseline sogar *senken*, siehe §4.5) und ein reines Ranking nach Betrag Schäden beider Vorzeichen gleich gewichtet.

Validierungs-Gate. Um sicherzustellen, dass die Isolation korrekt implementiert ist, dass also tatsächlich nur die Zielschicht quantisiert wird und alle übrigen Modellteile numerisch mit dem FP32-Referenzmodell übereinstimmen, wird für jede Kombination eine automatisierte Konsistenzprüfung durchgeführt, deren Ergebnis in `accuracy_mismatch.csv` protokolliert wird. Diese Datei muss für jede berichtete Kombination leer sein.

Warum Loss die natürliche Zielgröße ist. Die Genauigkeit ist stückweise konstant (eine Vorhersage ist entweder richtig oder falsch) und daher unempfindlich gegenüber kleinen Verschiebungen der Ausgabewahrscheinlichkeiten. Die Hessian-Matrix ist per Definition die Krümmung des Loss (§2.2), und Gleichung (1) sagt den Loss-Anstieg vorher, nicht einen Genauigkeitsverlust. Ein direkter Test der HAWQ-V2-Motivation muss daher konsequent denselben Loss als Schaden-Ziel verwenden, für den die Taylor-Entwicklung überhaupt aufgestellt wurde. Loss ist entsprechend als Damage-Metrik dieser Analyse gesetzt, nicht als eine von mehreren gleichwertigen Optionen. Ergänzend wird geprüft, ob die Kernbefunde robust gegenüber dieser Wahl sind, indem dieselbe Analyse zusätzlich mit genauigkeitsbasiertem Schaden wiederholt wird (§5.12); beide Versionen führen qualitativ zu denselben Schlussfolgerungen.

4.4 Drei Sensitivitäts-Scores

Verglichen werden drei Scores gegen `abs_loss_damage(l)`:

$$S_{\text{raw}}(l) = \text{Tr}(H_l), \quad (3)$$

$$S_{\text{pert}}(l) = \|W_l - Q(W_l)\|^2, \quad (4)$$

$$S_{\text{hawq}}(l) = \text{Tr}(H_l) \cdot \|W_l - Q(W_l)\|^2. \quad (5)$$

S_{raw} ist die rohe kanonische Krümmung (kanonische fusionierte Basis, gemessen auf dem FP32-Modell, unnormalisierte Hutchinson-Trace-Summe, *nicht* durch die Parameterzahl geteilt). S_{pert} ist die quadrierte Störungsgröße, die durch PoT-Rundung entsteht. S_{hawq} ist das HAWQ-V2-Produkt [2].

Begründung der Auswahl. S_{raw} und S_{pert} sind exakt die zwei multiplikativen Faktoren, aus denen S_{hawq} gebildet wird, und keiner der beiden Faktoren enthält Information über den jeweils anderen. Der Vergleich aller drei Scores testet daher direkt, ob der Schaden von der Krümmung, von der Störungsgröße oder tatsächlich vom Produkt beider getrieben wird, statt gegen beliebige externe Alternativen zu testen.

Parameter-Normalisierung. S_{raw} wird bewusst *nicht* durch die Parameterzahl der Schicht normalisiert, was in Teilen der Literatur übliche Praxis ist, um große und kleine Schichten fair zu vergleichen. Eine parameter-normalisierte Variante verbessert die Korrelation zwar in 5 von 6 Architektur×Datensatz-Kombinationen, verschlechtert sie aber gerade bei ResNet-50/CIFAR10 deutlich (Spearman- ρ fällt von 0.36 auf 0.16; Details in §5.6). Da dies dem HAWQ-Stil des unnormalisierten Scores entspricht und keine der beiden Varianten universell überlegen ist, wird konsistent die unnormalisierte Form als Hauptresultat berichtet.

Evaluationsmetriken. Jeder Score wird auf drei Arten ausgewertet: (1) die Spearman-Rangkorrelation zwischen den Score-Werten und `abs_loss_damage` über alle Schichten, mit 95%-Bootstrap-Konfidenzintervall ($B = 2000$ Resamples, Ziehen von Schichten mit Zurücklegen); (2) der Rang, den der Score der Schicht mit dem tatsächlich größten Schaden zuweist (Rang 1 = perfekt); (3) die Überlappung zwischen den top- k^* vom Score als am schädlichsten eingestuft Schichten und den top- k^* tatsächlich am meisten geschädigten Schichten.

4.4.1 Größennormalisierung der Top- k -Auswertung

Da die drei Architekturen eine Größenordnung an Schichtzahl umspannen ($n = 6, 21, 54$), wäre ein fester Top-5-Schwellwert für das `cnn` fast das gesamte

Sechs-Phasen-Analyse-Pipeline (Phase 4 = Kernanalyse, AXIOM-Paper)

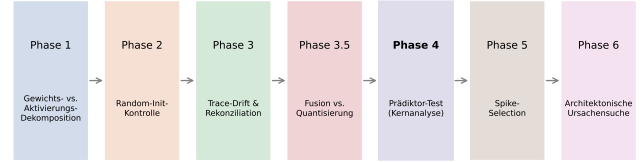


Abbildung 1: Sechs-Phasen-Analyse-Pipeline. Phase 4 (Prädiktor-Test) ist die im Workshop-Paper berichtete Kernanalyse; alle übrigen Phasen liefern Kontext, Kontrollen oder Erweiterungen dazu. Erzeugt von `notebooks/12_pipeline_overview.ipynb`.

Netz, für ResNet-50 dagegen nur ein kleiner Ausschnitt und damit kein über die Architekturen hinweg vergleichbarer Maßstab. Verwendet wird daher $k^* = \lceil 0.1n \rceil$ (konkret $k^* = 1, 3, 6$ für `cnn`, ResNet-18, ResNet-50), sodass alle drei Architekturen und alle drei Scores am gleichen Top-Dezil-Maßstab gemessen werden.

4.5 Gewichts- vs. Aktivierungs-Dekomposition (Phase 1)

Um F3 zu beantworten, wird der Gesamt-Genauigkeitsverlust einer Kombination in drei Konfigurationen zerlegt: (a) volle Quantisierung (Gewichte und Aktivierungen), (b) nur Gewichte quantisiert (Aktivierungen bleiben FP32), (c) nur Aktivierungen quantisiert (Gewichte bleiben FP32). Für (c) werden Beobachter zur Aktivierungs-Kalibrierung auf einem Kalibrierungs-Batch eingesetzt, wie bei Standard-PTQ üblich. Aus dem Vergleich der resultierenden Genauigkeiten werden `weight_loss_pts` und `activation_loss_pts` abgeleitet (Genauigkeitsverlust in Prozentpunkten gegenüber FP32), und die `dominant_source` wird als die Konfiguration mit dem größeren Betrag bestimmt. Ergänzend werden die Aktivierungswertebereiche pro Schicht sowie der darin auftretende Ausreißerfaktor (Verhältnis von Maximalwert zu 99. Perzentil) analysiert.

4.6 Weitere Analysephasen

Über Phase 1 und die Kernanalyse aus §4.3–4.4 (im Ergebnisteil als „Phase 4“ geführt) hinaus umfasst die Pipeline vier weitere Phasen, deren Methodik hier kurz skizziert und deren Ergebnisse in §5 berichtet werden. Abbildung 1 zeigt die sechs Phasen im Überblick.

Phase 2: Random-Init-Kontrolle (§5.3). Um

zu unterscheiden, ob eine beobachtete Krümmungsspitze in einer Schicht „architektonisch“ (bereits bei zufälliger Initialisierung vorhanden) oder „gelernt“ (erst durch Training entstanden) ist, wird die PyHessian-Trace auf drei unabhängig zufällig initialisierten, untrainierten Modellen pro Architektur (drei Init-Seeds) gemessen und das resultierende Trace-Profil mit dem des trainierten FP32-Modells verglichen.

Phase 3: Trace-Drift und Rekonziliation (§5.4). Da frühere, unterschiedlich konfigurierte Läufe im Projektverlauf teils stark abweichende Trace-Werte für dieselbe nominelle Größe produziert hatten, wird ein systematischer Sweep über die in §4.2 genannten Konfigurationsknöpfe durchgeführt (Basis, Loss-Reduktion, Proben-Anzahl, Datenquelle, Gerät), um zu prüfen, welche Konfiguration, falls überhaupt eine, die historischen Werte reproduziert.

Phase 3.5: Fusion vs. Quantisierung (§5.5). Die Batch-Normalisierungs-Fusion (das Verschmelzen von BN-Parametern in die vorangehende Faltungsschicht, Standardpraxis vor der Quantisierung) verändert die effektiven Gewichte einer Schicht bereits *vor* jeder Quantisierung. Um den Fusion-Effekt vom eigentlichen PoT-Quantisierungseffekt zu trennen, werden die Trace-Werte in unfusionierter FP32-Basis, fusionierter FP32-Basis und PTQ-Basis verglichen, und eine Schicht wird als „quant_induced“ definiert, wenn die PTQ-Trace deutlich über die bereits durch Fusion erhöhte FP32-Trace hinausgeht, andernfalls als „fp32_intrinsic“.

Phase 5: Spike-Selection (§5.7). Verglichen wird die Schicht mit der höchsten Trace („Trace-Spike“) direkt mit der Schicht mit dem größten gemessenen Schaden („Damage-Spike“, aus Phase 4/§4.3) pro Modell, um zu prüfen, ob die naive Heuristik „höchste Krümmung = größter Schaden“ zutrifft.

Phase 6: Architektonische Ursachensuche (§5.8). Für die jeweilige Trace-Spike-Schicht jedes Modells wird versucht, die beobachtete Krümmungserhöhung über die KFAC-Faktorisierung (§2.4) in einen Eingabe-getriebenen (A) und einen Gradienten-getriebenen (G) Anteil zu zerlegen und die vorhergesagte Pro-Parameter-Trace mit der tatsächlich gemessenen zu vergleichen. Ergänzend werden größen-kontrollierte Residuen berechnet (Korrelation von Trace-Ausreißern mit strukturellen Deskriptoren wie Fan-in, Fan-out und Output-Map-Größe, unter Kontrolle der Parameterzahl).

4.7 Deployment-Evaluation

Um zu prüfen, ob der theoretische Bitshift-Vorteil von PoT (§2.1) auf realer Hardware tatsächlich in eine kürzere Inferenzzeit übergeht, werden zwei Deployment-Pfade evaluiert. Der erste Pfad ist `torchao 0.17` auf einer NVIDIA A100-GPU (Compute Capability 8.0). Dabei werden `torchao` INT8-Konfigurationen (`Int8DynamicActivationInt8WeightConfig` für die dynamische Aktivierungsquantisierung, `IntxWeightOnlyConfig` für die reine Gewichtsquantisierung) auf alle sechs Kombinationen angewendet und die Durchsatz-Zeit über eine Reihe von Batch-Größen vermessen, jeweils gegen die FP32-Baseline auf derselben GPU. Der zweite Pfad ist ein CPU-Pfad über den `fbgemm`-Backend [10] von PyTorchs `eager-mode-PTQ`, der auf x86-Hardware mit AVX-512-VNNI-Unterstützung (Zielsystem: AMD Ryzen 7 7700X, Zen 4) tatsächlich quantisierte Conv2d- und Linear-Kernel einsetzt, im Gegensatz zum `torchao`-GPU-Pfad, dessen Einschränkungen in §5.11 dokumentiert werden.

5 Ergebnisse

5.1 Kern-Genauigkeiten

Tabelle 1 fasst die FP32-, PTQ- und QAT-Genauigkeit für alle sechs Architektur×Datensatz-Kombinationen zusammen (Quelle: `pipeline_summary.csv`, `ptq_summary.csv`, `qat_summary.csv`). PTQ verursacht durchweg einen Genauigkeitsverlust gegenüber FP32, der auf ImageNet100 deutlich größer ausfällt als auf CIFAR10 (etwa `cnn`: -17.98 vs. -10.22 Prozentpunkte). QAT gleicht diesen Verlust nicht nur aus, sondern übertrifft in jeder Kombination sogar die FP32-Baseline, am deutlichsten beim `cnn` auf ImageNet100 (FP32 43.04 %, QAT 51.18 %, +8.14 Punkte gegenüber FP32, +26.12 Punkte gegenüber PTQ). Dieser Effekt ist konsistent mit dem simulierten Quantisierungsrauschen als impliziter Regularisierung während des Trainings.

Auffällig ist, dass PTQ auf CIFAR10 (kleine 32×32 -Eingaben) für die ResNets moderat ausfällt (-1.4 bis -1.6 Punkte), auf ImageNet100 aber massiv (-8.8 bis -16.0 Punkte). Diese Asymmetrie deutet bereits methodisch an, dass Datensatz und Auflösung die Quantisierungsempfindlichkeit stark mitbestimmen, ein Muster, das sich durch die gesamte Analyse zieht (§5.6).

Tabelle 1: FP32-, PTQ- und QAT-Validierungsgenauigkeit (%) für alle sechs Kombinationen, mit PTQ-Verlust und QAT-Verbesserung gegenüber PTQ (Prozentpunkte). Quelle: `pipeline_summary.csv`.

Datensatz	Modell	FP32	PTQ	QAT	PTQ-Verlust	QAT-Erholung
CIFAR10	CNN	81.30	71.08	84.46	-10.22	+13.38
CIFAR10	ResNet-18	84.18	82.76	90.51	-1.42	+7.75
CIFAR10	ResNet-50	81.14	79.54	86.80	-1.60	+7.26
ImageNet100	CNN	43.04	25.06	51.18	-17.98	+26.12
ImageNet100	ResNet-18	80.06	71.30	82.52	-8.76	+11.22
ImageNet100	ResNet-50	80.02	64.06	83.22	-15.96	+19.16

Tabelle 2: Gewichts- vs. Aktivierungs-Dekomposition, CIFAR10 (Prozentpunkte gegenüber FP32; negativ = Verbesserung). Quelle: `activation_decomposition.csv`.

Modell	Stufe	Gewicht	Akt.	dominant
CNN	PTQ	10.24	-0.07	Gewicht
CNN	QAT	-3.22	3.11	Aktivierung
ResNet-18	PTQ	1.41	0.09	Gewicht
ResNet-18	QAT	-6.36	-5.48	Aktivierung
ResNet-50	PTQ	1.90	-0.23	Gewicht
ResNet-50	QAT	-5.50	-3.72	Aktivierung

5.2 Phase 1: Gewichts- vs. Aktivierungs-Dekomposition

Tabelle 2 zerlegt den PTQ- und QAT-Genauigkeitsverlust in einen Gewichts- und einen Aktivierungsanteil (Quelle: `activation_decomposition.csv`; ausschließlich für CIFAR10 verfügbar). Unter PTQ dominiert in allen drei Architekturen der Gewichtsanteil den Genauigkeitsverlust deutlich, am stärksten beim `cnn` (10.24 gegenüber -0.07 Punkten), am moderatesten bei ResNet-50 (1.90 gegenüber -0.23 Punkten). Unter QAT kehrt sich das Bild um. Der isolierte Aktivierungspfad verliert relativ stärker, während der isolierte Gewichtspfad die FP32-Baseline übertrifft (negativer „Verlust“, das heißt eine Verbesserung), konsistent mit der QAT-Verbesserung aus Tabelle 1.

Konsequenz für die Hessian-Analyse. Da die Hessian-basierte Sensitivitätsanalyse (§4.3–4.4) explizit nur Gewichtsschaden erfasst, ist ihr Anwendungsbereich durch diese Dekomposition abgesteckt. Unter PTQ, wo die Gewichte den Verlust dominieren, adressiert die Hessian-Analyse tatsächlich den Hauptmechanismus. Eine Übertragung auf die QAT-Sensitivität wäre dagegen fragwürdig, da dort die Aktivierungen relativ stärker zum Verlust beitragen. Dies ist ein Grund, warum die Hauptanalyse auf PTQ fokussiert wird (F3, teilweise beantwortet; ImageNet100-Daten zu dieser Dekomposition liegen

Tabelle 3: `conv1`-Trace bei Zufallsinitialisierung vs. trainiert (FP32, unfusioniert). Klassifikation laut `random_init_comparison.csv`.

Modell	Random ($\mu \pm \sigma$)	Trainiert	Klasse
CNN	0.0024 ± 0.0001	65.87	gelernt
ResNet-18	7.22 ± 1.19	17.35	gelernt
ResNet-50	74.4 ± 68.6	15.55	gelernt

nicht vor, siehe §6.6).

Ergänzend zeigt die Aktivierungs-Ablationsanalyse (`activation_ablation.csv`), dass das sukzessive Zurücksetzen der Schichten mit dem höchsten Aktivierungs-Ausreißerfaktor (bis zu Faktor $25.1\times$ bei ResNet-50-layer4.1.conv3) auf FP32 nur geringe Genauigkeit zurückgewinnt (etwa +0.16 Punkte für ResNet-50/PTQ nach Ausschluss der fünf am stärksten ausreißenden Schichten). Aktivierungs-Ausreißer sind zwar messbar vorhanden (Wertebereichsfaktor `outlier_factor` typischerweise $3\text{--}5\times$ bei Faltungsschichten, `activation_ranges.csv`), tragen aber unter PTQ nicht dominant zum Gesamtschaden bei, konsistent mit dem in Tabelle 2 beobachteten „dominant=Gewicht“-Befund für PTQ.

5.3 Phase 2: Random-Init-Kontrolle

Tabelle 3 vergleicht die Trace der `conv1`-Schicht bei zufälliger Initialisierung (Mittelwert $\pm$ Standardabweichung über drei Init-Seeds) mit der Trace desselben Layouts nach Training (FP32, unfusionierte Basis; Quelle: `random_init_comparison.csv`, `random_init_summary.csv`).

In allen drei Modellen ist `conv1` als „gelernt“ klassifiziert. Beim `cnn` liegt die trainierte Trace um den Faktor ~ 27500 über der Zufalls-Trace, was die extremste der drei Verschiebungen darstellt. Bei ResNet-18 beträgt der Faktor ~ 2.4 , und bei ResNet-50 liegt die trainierte Trace unterhalb des mit sehr hoher Streuung behafteten Zufalls-

Mittelwerts, wird aber dennoch als „gelernt“ eingestuft, da die Elevation relativ zum medianen Trace-Wert des jeweiligen Modells nach Training deutlich zunimmt (`elev_over_median_random`= 0.10 vs. `elev_over_median_trained`= 1.02). Die Kernaussage ist über alle drei Architekturen konsistent. Die charakteristische `conv1`-Krümmungsspitze, die für die Prädiktor-Analyse in §5.6 zentral ist, entsteht durch Training und ist *nicht* bereits eine Eigenschaft der Netzarchitektur bei Initialisierung. Für ResNet-50 zeigt der volle Trace-Profil-Vergleich (alle Schichten, nicht nur `conv1`) zusätzlich eine signifikante Spearman-Korrelation zwischen Random- und Trainiert-Profil ($\rho = 0.60$, $p < 0.001$) mit dem stärksten Ausreißer bei `layer4.1.conv2` (Elevation Random= 38.8×, Trainiert= 7.8×). Dies ist ein Hinweis darauf, dass bei ResNet-50 zusätzlich zur `conv1`-Dynamik auch architektonische, bereits bei Init vorhandene Effekte tiefer im Netz eine Rolle spielen, während beim ResNet-18-Profil keine signifikante Korrelation besteht ($\rho = 0.09$, $p = 0.70$).

Diese tiefenabhängige Struktur bei Zufallsinitialisierung ist qualitativ konsistent mit der Mean-Field-Vorhersage von Karakida, Akaho und Amari [9] (§2.4). Das Training kehrt dieses anfängliche Tiefenprofil für `conv1` systematisch um, indem es dessen Krümmung relativ zum Rest des Netzes anhebt.

5.4 Phase 3: Trace-Drift und Rekonziliation

Im Projektverlauf entstanden für dieselbe nominelle Größe, die PTQ-Trace von ResNet-50-`conv1`, deutlich unterschiedliche historische Werte, abhängig davon, welche nicht dokumentierte Konfiguration ein früherer Lauf verwendete, nämlich 13161.8 in einem Lauf und 1030.9 in einem späteren. Die kanonische Neu-Messung (§4.2) ergibt 97.76 ± 6.27 (Mittelwert±Standardabweichung über drei Proben-Seeds; `canonical_traces.csv`), was Verhältnissen von 134.6× bzw. 10.5× zu den historischen Werten entspricht.

Um diese Diskrepanzen zu erklären, wird ein systematischer Sweep über Basis (fusioniert/unfusioniert), Loss-Reduktion (Mittelwert/Summe), Proben-Anzahl, Datenquelle und Recheneinheit durchgeführt (`drift_diagnosis.csv`). Nur der FP32-Wert aus Lauf 1 lässt sich reproduzieren, und zwar durch die unfusionierte Basis mit einem Residuum von 2.8%. Keine getestete Konfiguration reproduziert die beiden historischen PTQ-Werte auch nur annähernd. Selbst die extremsten getesteten Varianten, also die Summenreduktion (14101 für FP32, 7742 für PTQ),

das kleinste Proben-Budget (`maxIter`= 20: 139.9 bzw. 82.5) oder ungeseedete Trainingsdaten mit Augmentation (286.0/184.4 bzw. 105.6/388.6), bleiben um mindestens eine Größenordnung von 13161.8 entfernt.

Dies wird als dokumentierter, abgeschlossener Befund berichtet und nicht als offene Frage. Die historischen PTQ-Werte lassen sich mit den verfügbaren Konfigurationsknöpfen nicht rekonstruieren, und plausible Ursachen sind ein damals nicht protokollierter Checkpoint-, Kalibrierungs- oder Codepfad-Unterschied, der außerhalb des hier gesweepeten Parameterraums liegt. Die praktische Konsequenz ist die in §4.2 eingeführte Disziplin. Ohne eine explizit dokumentierte, fixierte Konfiguration sind Krümmungs-basierte Sensitivitätswerte zwischen Läufen, und potenziell zwischen Publikationen, nicht direkt vergleichbar.

5.5 Phase 3.5: Fusion vs. Quantisierung

Batch-Normalisierungs-Fusion allein erhöht die `conv1`-Trace bereits erheblich, bevor überhaupt quantisiert wird (Verhältnis fusioniert/unfusioniert, FP32, `canonical_traces.csv`): 11.7× bei ResNet-50 (174.5 vs. 14.9), 10.7× bei ResNet-18 (182.1 vs. 17.0), 7.3× beim `cnn` (460.5 vs. 63.1). Tabelle 5 setzt dies in Bezug zur zusätzlichen Amplifikation durch PTQ selbst (`conv1_ptq_amplification` = $\text{Tr}_{\text{PTQ}}/\text{Tr}_{\text{FP32,fused}}$, sowie deren Rang unter allen Schichten des Modells; Quelle: `quant_induced_summary.csv`).

Nur beim `cnn` übersteigt die PTQ-Trace die bereits fusionierte FP32-Trace nennenswert (Faktor 1.15); bei ResNet-18 und ResNet-50 liegt die PTQ-Trace *unterhalb* der fusionierten FP32-Trace (Faktoren 0.95 und 0.56), und in beiden Fällen liegt `conv1` in der PTQ-Amplifikations-Rangfolge nahe am unteren Ende (Rang 19 von 21 bzw. 51 von 54). Die dramatische `conv1`-Erhöhung, die in §5.3 beobachtet wurde, ist demnach primär ein Fusions-Effekt (bzw. ein Trainingseffekt, siehe Phase 2) und nicht primär ein Quantisierungs-induzierter Effekt, mit der Ausnahme des `cnn`, wo PTQ selbst eine zusätzliche, wenn auch moderate Verstärkung beiträgt. Diese Unterscheidung ist wichtig für die korrekte Interpretation von Tabelle 6 in §5.6. Die hohe `conv1`-Krümmung, die dort als Prädiktor-Signal dient, entsteht überwiegend *vor* der eigentlichen PoT-Rundung.

Tabelle 4: Rekonziliations-Ledger für ResNet-50-conv1: historische Werte gegen die kanonische Neu-Messung. Quelle: `trace_reconciliation_ledger.csv`.

Größe	Alter Wert	Kanonisch	Verhältnis	Erklärung
conv1 FP32 (Lauf 1)	13.87	14.90	0.93×	unfusionierte Basis (Residuum 2.8 %)
conv1 FP32 (Lauf 2)	11.79	14.90	0.79×	ungeklärt
conv1 PTQ (Lauf 1)	13161.8	97.76	134.6×	ungeklärt
conv1 PTQ (Lauf 2)	1030.9	97.76	10.5×	ungeklärt
conv1 Elevation FP32	14.0	11.73	1.19×	ungeklärt

Tabelle 5: Fusion- und PTQ-Amplifikation für conv1. Rang = Position von conv1 in der Rangfolge aller Schichten nach PTQ-Amplifikation (1 = am stärksten amplifiziert). Quelle: `quant_induced_summary.csv`.

Modell	Fusion	PTQ-Amp.	Rang	Verdikt
CNN	7.3×	1.15	6/6	quant_induced
ResNet-18	10.7×	0.95	19/21	fp32_intrinsic
ResNet-50	11.7×	0.56	51/54	fp32_intrinsic

5.6 Phase 4: Prädiktor-Test (Kernergebnis)

Dies ist die zentrale Analyse des Projekts und die Grundlage des Workshop-Papers. Tabelle 6 zeigt für alle sechs Kombinationen (PTQ, loss-basierter Schaden), welchen Rang S_{raw} der Schicht mit dem tatsächlich größten Schaden zuweist, sowie die top- k^* -Überlappung aller drei Scores (Quelle: `weight_ablation_loss_damage_correlation.csv`, `normalized_ranks_loss.csv`).

S_{raw} erreicht eine Rang-1-Identifikation in 3 von 6 Kombinationen und wird in der top- k^* -Überlappung von keinem der beiden anderen Scores übertroffen. In beiden `cnn`-Zeilen liegt es gleichauf (dort erlaubt $k^* = 1$ keinem Score, die einzige wahre Top-Schicht zu treffen), in den übrigen vier Kombinationen ist es strikt überlegen. S_{pert} erreicht in keiner Kombination Rang 1 und versenkt die wahre Top-Schicht regelmäßig weit hinten in der Rangfolge, insbesondere wenn diese eine Faltungsschicht mit kleinem Fan-in ist (`conv1, layer1.1.conv1`). S_{hawq} erreicht ebenfalls nie Rang 1 und übertrifft S_{raw} s top- k^* -Überlappung in keiner Kombination. ResNet-50/CIFAR10 ist der Fall, in dem S_{raw} selbst am stärksten versagt (Rang 40 von 54), konsistent damit, dass der Schaden dort am diffusesten über alle 54 Schichten verteilt ist. Hier erreicht keiner der drei Scores mehr als ein Drittel der wahren top- k^* -Menge. Von den sechs S_{raw} -Korrelationen in der Tabelle schließt nur eine, ResNet-50/CIFAR10, das 95%-Bootstrap-CI von Null aus; die übrigen fünf sind richtungskonsistent positiv, aber bei dieser Stichprobengröße ($n = 6$ bis 54 Schichten) statistisch nicht gesichert, ein Punkt, der in §5.6 unten weiter vertieft wird.

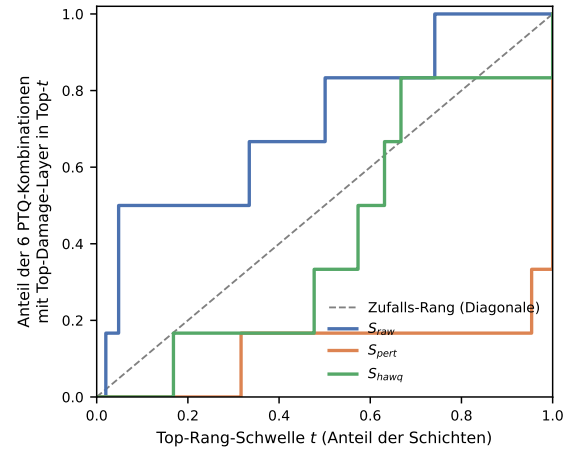


Abbildung 2: Kumulative Rang-Kurven: Anteil der 6 PTQ-Kombinationen, in denen die wahre Top-Damage-Schicht innerhalb der obersten t -Schichten (x-Achse) liegt, pro Score. Graue Diagonale = Zufalls-Rang-Erwartung. Erzeugt von `notebooks/16_full_ranking_curves.ipynb`.

Abbildung 2 bündelt diese sechs Einzelfälle zu einer über die Architekturen normalisierten Zusammenfassung. Für eine Rang-Schwelle $t \in [0, 1]$ zeigt sie den Anteil der sechs PTQ-Kombinationen, in denen der jeweilige Score die wahre Top-Damage-Schicht innerhalb der obersten t -Schichten platziert. S_{raw} dominiert über den gesamten Schwellwertbereich, erreicht bereits bei $t = 0.05$ eine Trefferquote von 3/6 und bei $t = 0.5$ von 5/6, während S_{pert} selbst bei $t = 0.5$ noch keine einzige Kombination erfasst (seine engste Platzierung liegt bei $t = 0.31$) und S_{hawq} dazwischen liegt.

Abbildung 3 zeigt die vollständige Score-Werts-vs-Schaden-Beziehung für alle sechs Kombinationen

Tabelle 6: Rang des wahren Top-Damage-Layers unter S_{raw} , dessen Spearman- ρ mit 95%-Bootstrap-CI, und top- k^* -Überlappung aller drei Scores, PTQ, loss-basiert. $k^* = 1, 3, 6$ für CNN/ResNet-18/ResNet-50. Fett = CI schließt Null aus. Quelle: `normalized_ranks_loss.csv`, `bootstrap_ci_spearman_loss.csv`.

Datensatz	Modell (n)	Top-Damage-Layer	S_{raw} -Rang	$S_{\text{raw}} \rho$ [95%-CI]	S_{raw} top- k^*	S_{pert} top- k^*	S_{hawq} top- k^*
ImageNet100	CNN (6)	conv1	3	0.54 [-1.00, 0.80]	0/1	0/1	0/1
ImageNet100	ResNet-18 (21)	conv1	1	0.10 [-0.43, 0.54]	1/3	0/3	0/3
ImageNet100	ResNet-50 (54)	conv1	1	0.13 [-0.17, 0.41]	1/6	0/6	0/6
CIFAR10	CNN (6)	conv1	2	0.77 [0.00, 1.00]	0/1	0/1	0/1
CIFAR10	ResNet-18 (21)	layer1.1.conv1	1	0.42 [-0.01, 0.75]	1/3	0/3	0/3
CIFAR10	ResNet-50 (54)	fc	40	0.36 [0.10, 0.58]	2/6	1/6	1/6

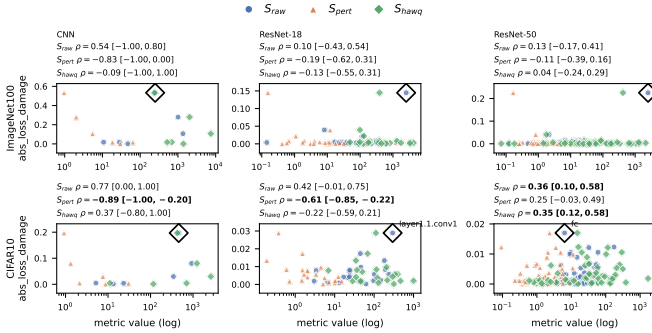


Abbildung 3: Score-Wert (log-Skala) vs. gemessener `abs_loss_damage`, PTQ; obere Zeile ImageNet100, untere Zeile CIFAR10, Spalten CNN/ResNet-18/ResNet-50. Offene Diamanten markieren die wahre Top-Damage-Schicht (beschriftet, wo nicht `conv1`). Jedes Panel-Inset zeigt Spearman- ρ mit 95%-Bootstrap-CI ($B = 2000$); fett = CI schließt Null aus. Erzeugt von `notebooks/06_main_facet_plot.ipynb`.

mit den zugehörigen Spearman-Korrelationen und Bootstrap-Konfidenzintervallen. Die volle Rangkorrelation ist insgesamt fragil. Von 18 Score×Kombination-Paaren sind 5 bei $p < 0.05$ signifikant (4, wenn ein Konfidenzintervall, das die Null ausschließt, als Kriterium verwendet wird), davon vier für S_{pert} bzw. S_{hawq} mit *negativem* Vorzeichen. S_{pert} korreliert in 5 von 6 Kombinationen negativ mit dem Schaden, entgegengesetzt zu S_{raw} s durchweg positivem Vorzeichen in denselben 5 Kombinationen.

Parameter-Normalisierung. Eine parameter-normalisierte Variante von S_{raw} (Division durch die Parameterzahl der Schicht) verbessert die Korrelation in 5 von 6 Kombinationen (etwa `cnn`/CIFAR10: $\rho = 0.77 \rightarrow 1.00$), verschlechtert sie aber bei ResNet-50/CIFAR10 deutlich ($\rho = 0.36 \rightarrow 0.16$; Quelle: `raw_trace_per_param_comparison_loss.csv`). Da kein Normalisierungsschema universell überlegen ist, bleibt die unnormalisierte Form das Hauptresultat (§4.4).

Metrik-Konkordanz. Tabelle 7 zeigt die paarweise Spearman-Korrelation zwischen den drei Score-Werten selbst (unabhängig vom Schaden-Ziel; Quelle: `predictor_concordance.csv`). Bei ResNet-50 sind S_{pert} und S_{hawq} nahezu identisch ($\rho = 0.84$ CIFAR10, $\rho = 0.83$ ImageNet100), sodass der Drei-Wege-Vergleich dort effektiv zu einem Zwei-Wege-Vergleich wird (S_{raw} gegen ein fast identisches $S_{\text{pert}}/S_{\text{hawq}}$ -Paar). Dass S_{hawq} s top- k^* -Überlappung bei ResNet-50 (Tabelle 6) S_{raw} nie übertrifft, ist somit größtenteils S_{pert} s Muster, das durch das Produkt hindurchgereicht wird, und kein unabhängiges drittes Ergebnis.

Die vorstehende Analyse ist auf PTQ skaliert; der direkte Vergleich mit der QAT-Krümmungsstruktur, der F1 beantwortet, folgt in §5.9.

5.7 Phase 5: Spike-Selection

Tabelle 8 vergleicht für CIFAR10 die Schicht mit der höchsten Krümmung im trainierten FP32-Modell („Trace-Spike“) mit der Schicht mit dem größten gemessenen Loss-Schaden („Damage-Spike“), Quelle `spike_selection.csv` (kanonische fusionierte Basis für die Trace-Spike-Bestimmung, konsistent mit `canonical_traces.csv`).

Unter der kanonischen fusionierten Basis stimmen Trace-Spike und Damage-Spike in 1 von 3 CIFAR10-Modellen überein, nämlich bei ResNet-18. Beim `cnn` liegt die Trace-Spitze bei `conv2`, während `conv1` am stärksten geschädigt wird; bei ResNet-50 liegt die Trace-Spitze bei `conv1` (Fusions-getrieben, §5.5), während `fc` am stärksten geschädigt wird. Auf ImageNet100 (ebenfalls `spike_selection.csv`) fällt die Übereinstimmung höher aus. ResNet-50 und ResNet-18 stimmen beide überein (`conv1` ist dort sowohl Trace- als auch Damage-Spike), nur das `cnn` weicht ab (Trace-Spitze `conv3`, Damage-Spitze `conv1`), womit 2 von 3 ImageNet100-Modellen übereinstimmen. Dieser Datensatz-Unterschied ist konsistent mit dem in §5.6 beobachteten Muster, dass `conv1` auf ImageNet100 durchweg die am stärksten

Tabelle 7: Paarweise Spearman- ρ zwischen den drei Score-Werten, PTQ. Quelle: `predictor_concordance.csv`.

Modell	Datensatz	$\rho(S_{\text{raw}}, S_{\text{pert}})$	$\rho(S_{\text{raw}}, S_{\text{hawq}})$	$\rho(S_{\text{pert}}, S_{\text{hawq}})$
CNN	ImageNet100	-0.71	0.60	0.09
CNN	CIFAR10	-0.54	0.71	0.03
ResNet-18	ImageNet100	-0.60	0.43	0.41
ResNet-18	CIFAR10	-0.55	0.45	0.43
ResNet-50	ImageNet100	0.40	0.78	0.83
ResNet-50	CIFAR10	0.48	0.86	0.84

Tabelle 8: Trace-Spike (höchste FP32-Trace, kanonische fusionierte Basis) vs. Damage-Spike (größter PTQ-Loss-Schaden), CIFAR10.

Modell	Trace-Spike	Damage-Spike	Übereinst.
CNN	<code>conv2</code>	<code>conv1</code>	nein
ResNet-18	<code>layer1.1.conv1</code>	<code>layer1.1.conv1</code>	ja
ResNet-50	<code>conv1</code>	<code>fc</code>	nein

geschädigte Schicht ist, auf CIFAR10 dagegen nicht immer.

Bemerkenswert ist zusätzlich, dass diese Übereinstimmung selbst basisabhängig ist. Verwendet man statt der kanonischen fusionierten FP32-Trace die Trace-Elevation relativ zur Zufallsinitialisierung aus der Random-Init-Vergleichsanalyse (`random_init_summary.csv`, unfusionierte Basis, §5.3), so ergibt sich für CIFAR10 eine andere Trace-Spike-Schicht pro Modell (ResNet-50: `layer4.1.conv2`; ResNet-18: `layer3.0.conv2`; `cnn:conv3`), und keines der drei Modelle stimmt mehr mit seinem Damage-Spike überein. Dieser Kontrast ist selbst ein Ergebnis. Ob die naive Heuristik „höchste Krümmung = größter Schaden“ zutrifft, hängt nicht nur vom Modell, sondern auch von der gewählten Trace-Konfiguration ab, was ein weiterer Beleg für die in §5.4 dokumentierte Konfigurations-Sensitivität ist.

5.8 Phase 6: Architektonische Ursachen-suche

Für die jeweilige Trace-Spike-Schicht jedes Modells (§5.7) wird versucht, die gemessene Pro-Parameter-Trace über die KFAC-Faktorisierung $\text{Tr}(F) \approx \text{Tr}(A) \cdot \text{Tr}(G)$ vorherzusagen (§2.4). Tabelle 9 zeigt, dass die Zerlegung in allen getesteten Fällen numerisch scheitert, da die vorhergesagte Pro-Parameter-Trace von der tatsächlich gemessenen um 3 bis über 8 Größenordnungen abweicht (Quelle: `spike_layer_kfac.csv`).

Die Abweichung ist für Faltungsschichten (`conv1`, `conv2`, `layer1.1.conv1`) systematisch größer (7–8

Größenordnungen) als für die vollverbundene `fc`-Schicht von ResNet-50 (gut 3 Größenordnungen). Dies ist ein Hinweis, dass die KFAC-Näherung, die ursprünglich für einzelne dichte Schichten entwickelt [16] und erst später auf Faltungsschichten erweitert wurde [5], für die hier betrachteten Faltungsschichten und deren spezifische Fan-in/Fan-out-Struktur (Tabelle 10) besonders schlecht kalibriert ist. Auch die aus der Faktorisierung abgeleiteten A - und G -Anteile (`spike_A_share`, `spike_G_share`) sind inkonsistent. Sie liegen wiederholt außerhalb des sinnvollen Bereichs $[0, 1]$ (etwa `cnn/conv1`: A -Anteil 16.6, G -Anteil -15.6), was auf ein numerisch instabiles Verhältnis zwischen den Faktoren $\text{Tr}(A)$ und $\text{Tr}(G)$ hindeutet und nicht auf eine sinnvolle additive Zerlegung.

Das KFAC-Scheitern wird als ehrliches Negativresultat gewertet. Die in §2.4 skizzierte Hoffnung, Krümmungsspitzen über eine Eingabe-/Gradienten-Zerlegung architektonisch zu erklären, lässt sich mit der verwendeten Implementierung im PoT-Kontext nicht einlösen. Die drei Hypothesen H1–H3 zur architektonischen Ursache (Fan-in-getrieben, Output-Map-Größe-getrieben, vorangehender- A -Faktor-getrieben; `spike_layer_attribution.csv`) konnten entsprechend nicht diskriminiert werden. Die Spalte `hypothesis_supported` ist für alle getesteten Schicht-Deskriptor-Paare „undetermined“, unter anderem weil der für eine Diskriminierung vorgesehene Auflösungs-kontrast zwischen CIFAR10 und ImageNet100 (`resolution_contrast_ratio`) für keine Kombination auswertbar war (durchweg NaN).

Tabelle 9: KFAC-Vorhersage vs. gemessene Pro-Parameter-Trace für die jeweilige Trace-Spike-Schicht, PTQ. Verhältnis = gemessen / vorhergesagt. Quelle: `spike_layer_kfac.csv`.

Modell	Schicht	Vorhergesagt	Gemessen	Verhältnis
CNN	conv1	3.6×10^{-9}	0.614	1.7×10^8
CNN	conv2	2.3×10^{-9}	0.069	3.0×10^7
ResNet-18	layer1.1.conv1	3.5×10^{-11}	0.0069	2.0×10^8
ResNet-50	conv1	3.7×10^{-9}	0.0566	1.5×10^7
ResNet-50	fc	1.3×10^{-7}	3.3×10^{-4}	2.5×10^3

Tabelle 10: Strukturelle Deskriptoren der Trace-Spike-Schichten. Quelle: `spike_layer_descriptors.csv`.

Modell	Schicht	Fan-in	Fan-out	#Param.
CNN	conv1	27	288	864
CNN	conv2	288	576	18432
ResNet-18	layer1.1.conv1	576	576	36864
ResNet-50	conv1	27	576	1728
ResNet-50	fc	2048	10	20480

5.9 QAT-Ergebnisse und PTQ-vs-QAT-Vergleich

Dieser Abschnitt beantwortet F1: Wie unterscheiden sich PTQ und QAT in der schichtweisen Verlust-Krümmung, und wie wirkt sich das auf die Score-vs-Schaden-Beziehung aus? Die AXIOM-Einreichung beschränkt sich auf PTQ; dieser Bericht ergänzt QAT auf beiden Ebenen.

Die Krümmungsstruktur selbst ändert sich unter QAT kaum. Vergleicht man die kanonische, fusionierte Hessian-Spur *pro Schicht* zwischen dem PTQ- und dem QAT-Checkpoint derselben Architektur×Datensatz-Kombination (Quelle: `weight_ablation_canonical_v2.csv` und dessen ImageNet100-Variante), ergibt sich eine durchweg sehr hohe Spearman-Korrelation zwischen beiden Trace-Profilen ($\rho = 0.95\text{--}1.00$ über alle sechs Kombinationen, $p < 0.001$ in jedem Fall). Welche Schichten relativ zueinander stark oder schwach gekrümmt sind, ist also weitgehend unabhängig davon, ob das Netz per PTQ nachträglich oder per QAT von Beginn an quantisierungsbewusst trainiert wurde. Das ist konsistent mit dem Befund aus §5.5, dass die dominanten Krümmungsspitzen primär architektonisch (Fusion, Fan-in) und nicht durch den spezifischen Trainingspfad bedingt sind.

Die Score-vs-Schaden-Beziehung bricht unter QAT dagegen fast vollständig zusammen. Abbildung 4 zeigt das QAT-Gegenstück zu Abbildung 3 (Bootstrap-Rho und CI hier direkt aus den QAT-Per-Layer-Werten neu be-

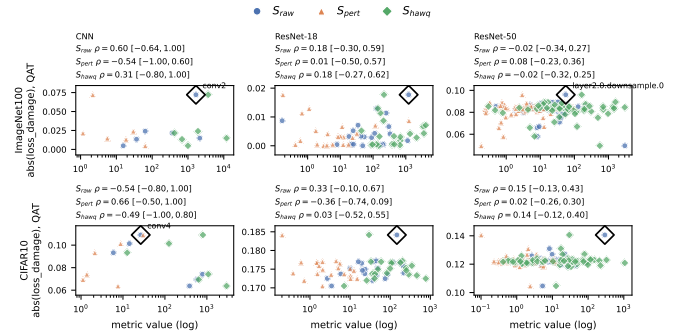


Abbildung 4: QAT-Gegenstück zu Abbildung 3: Score-Wert (log-Skala) vs. `abs_loss_damage`, QAT. Bootstrap-CI direkt aus den QAT-Werten berechnet ($B = 2000$, Seed 42). Erzeugt von `notebooks/14_qat_facet_plot.ipynb`.

rechnet, da `bootstrap_ci_spearman_loss.csv` nur PTQ abdeckt; Methodik identisch zu §4.4, $B = 2000$, Seed 42). Von den 18 loss-basierten QAT-Score×Kombination-Paaren ist *keines* bei $p < 0.05$ signifikant (kleinster p -Wert 0.10, ResNet-18/CIFAR10, S_{pert}); das ist sogar schwächer als das eine genauigkeitsbasierte QAT-Ergebnis, das im ursprünglichen Berichtsentwurf notiert wurde (ResNet-18/ImageNet100, S_{pert} , $\rho = -0.47$, $p = 0.031$; Quelle: `weight_ablation_canonical_correlation_v2.csv`).

Der Grund liegt in §5.2: Unter QAT verbessert die isolierte Quantisierung einer Schicht den Loss häufig gegenüber der FP32-Baseline (negativer Schaden), was Vorzeichen und Größenordnung des Zielwerts gegenüber PTQ grundlegend verändert. Die auf PTQ kalibrierte Score-Intuition, dass hohe Krümmung mit hohem Schaden einhergeht, überträgt sich damit nicht auf ein Regime, in dem „Schaden“ teils gar keiner ist. Antwort auf F1: PTQ und QAT reformen die *relative* Krümmungsstruktur eines Netzes kaum, sie reformen aber grundlegend, was diese Krümmung über den tatsächlichen Quantisierungseffekt aussagt.

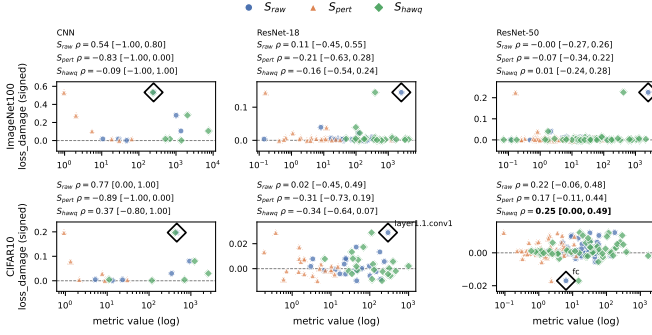


Abbildung 5: Wie Abbildung 3, jedoch mit vorzeichenbehaftetem `loss_damage` (PTQ) auf der y-Achse; gestrichelte Linie bei Null. Erzeugt von `notebooks/15_signed_damage_plot.ipynb`.

5.10 Signed vs. Absolute Damage

Die Kernanalyse (§5.6) rankt Schichten nach `abs_loss_damage`, da ein reines Vorzeichen-Ranking Schäden beider Richtungen ungleich behandeln würde (§4.3). Abbildung 5 zeigt zur Einordnung dasselbe PTQ-Raster wie Abbildung 3, jedoch mit dem *vorzeichenbehafteten* `loss_damage` auf der y-Achse.

Unter PTQ ist der Vorzeichenwechsel selten, aber nicht vernachlässigbar: von den 162 Schicht-Messungen über alle sechs Kombinationen sind 49 (30%) negativ, das heißt, die isolierte PTQ-Quantisierung dieser Schichten senkt den Loss geringfügig unter die FP32-Baseline (Quelle: `weight_ablation_loss_damage.csv`). Da diese negativen Werte betragsmäßig überwiegend klein sind, ändert sich die Identität der jeweils am stärksten geschädigten Schicht zwischen vorzeichenbehafteter und Betrags-Rangfolge in keiner der sechs Kombinationen. Der $|\cdot|$ -Betrag ist damit für das Ranking die richtige Wahl (§4.3), während die vorzeichenbehaftete Ansicht zusätzlich zeigt, dass „Schaden“ unter PTQ keine Einbahnstraße ist, ein Muster, das unter QAT (§5.9) noch deutlich ausgeprägter auftritt.

5.11 Deployment-Ergebnisse

Tabelle 11 zeigt die gemessene mittlere Laufzeit-Beschleunigung (INT8 relativ zu FP32, über eine Reihe von Batch-Größen gemittelt) auf einer NVIDIA A100-GPU für zwei torchao-0.17-Konfigurationen: die reine Gewichtsquantisierung (`int8_weight_only`) und die dynamische Aktivierungsquantisierung (`int8_dynamic_act`). Quelle ist ein projektinternes Diagnose-Protokoll (`int8_perf_diagnosis.txt`), das für jede der sechs Kombinationen und beide Quantisierungsstufen (PTQ/QAT-Checkpoint als Ausgangspunkt der Deployment-Konvertierung) einen vollständigen

Batch-Größen-Sweep protokolliert.

Die dynamische Aktivierungsquantisierung ist in jeder Kombination *langsamer* als FP32 (Faktor 0.21–0.67, entsprechend 1.5–4.8× langsamer). Der Grund ist strukturell und in der Diagnose dokumentiert. `torchao 0.17s` Standard-Filterfunktion für `Int8DynamicActivationInt8WeightConfig` erfasst ausschließlich `nn.Linear`-Schichten (`aten.linear/aten.mm`), nicht `aten.conv2d`; die einzige Conv2d-fähige Konfiguration, `IntxWeightOnlyConfig`, dequantisiert bei jedem Vorwärtsdurchlauf zurück auf FP32 und führt eine gewöhnliche FP32-Faltung aus. In dieser torchao-Version existiert schlicht kein optimierter INT8-Faltungskernel. Der moderate Geschwindigkeitsgewinn von `int8_weight_only` (1.05–1.19×) stammt entsprechend nicht aus beschleunigter Faltungsberechnung, sondern daraus, dass diese Konfiguration die pro Aufruf anfallenden Aktivierungs-Quantisierungskosten der dynamischen Variante vermeidet, während sie dennoch den Gewichtsspeicher reduziert.

CPU-Pfad (`fbgemm`). Der `fbgemm`-Backend von PyTorchs eager-mode-PTQ implementiert, im Gegensatz zum torchao-GPU-Pfad, echte quantisierte Conv2d- und Linear-Kernel für x86-Hardware mit AVX-512-VNNI (Zielsystem: AMD Ryzen 7 7700X, Zen 4). Die projektinternen Logs bestätigen, dass dieser Pfad für alle sechs Kombinationen ausgeführt wurde. Schichten werden nachweislich in echte quantisierte Conv2d- und Linear-Module konvertiert („confirmed quantized“) und auf 50 Kalibrierungs-Batches kalibriert. Ein vollständiger Durchsatz-/Genauigkeits-Sweep mit finaler CSV-Zusammenfassung (analog zu Tabelle 11) wurde in den verfügbaren Läufen jedoch nicht bis zum Abschluss protokolliert, denn eine `fbgemm_summary.csv` existiert in keinem der Ergebnisverzeichnisse. Der CPU-Pfad wird daher qualitativ als mechanistisch tragfähig berichtet (echte INT8-Kernel greifen nachweislich), ohne einen quantitativen Speedup zu behaupten, der sich nicht mit einer Quelle belegen lässt.

Speicherreduktion. INT8-Gewichte benötigen ein Byte pro Wert gegenüber vier Byte bei FP32, also ein theoretisches Maximum von 4× reiner Gewichtsspeicherreduktion. Ein tatsächlich gemessener Speicherreduktionsfaktor (etwa über nicht-quantisierte Anteile wie Bias- oder BN-Statistiken hinweg) wurde in keinem der verfügbaren Protokolle festgehalten, weshalb nur der theoretische obere Grenzwert und keine gemessene Zahl berichtet wird.

Gesamtbefund. Der PoT-Bitshift-Vorteil aus §2.1 übersetzt sich auf der getesteten GPU-Hardware

Tabelle 11: Mittlere INT8-vs-FP32-Laufzeit-Beschleunigung auf NVIDIA A100 (torchao 0.17), gemittelt über Batch-Größen 1–32. Werte > 1 = schneller als FP32. Quelle: `int8_perf_diagnosis.txt`.

Datensatz	Modell	wt.-only	dyn.-akt.
CIFAR10	CNN	1.05	0.21
CIFAR10	ResNet-18	1.19	0.56
CIFAR10	ResNet-50	1.12	0.65
ImageNet100	CNN	1.16	0.57
ImageNet100	ResNet-18	1.12	0.61
ImageNet100	ResNet-50	1.07	0.67

Tabelle 12: FP32-, PTQ- und QAT-Durchsatz (Bilder/s) in PyTorchs eager-mode-PTQ, ohne Backend-Kernel. Quelle: `ptq_summary.csv`, `qat_summary.csv`.

Datensatz	Modell	FP32	PTQ	QAT
CIFAR10	CNN	1027.6	397.1	390.4
CIFAR10	ResNet-18	306.2	79.5	37.7
CIFAR10	ResNet-50	133.6	27.6	13.8
ImageNet100	CNN	861.6	322.0	314.4
ImageNet100	ResNet-18	292.9	74.9	74.7
ImageNet100	ResNet-50	127.8	26.5	13.7

nicht in einen gemessenen Laufzeitvorteil. Im Gegenteil, die naheliegendste torchao-Konfiguration ist deutlich langsamer als FP32. Der CPU-Pfad mit echten INT8-Kernen ist mechanistisch der richtige Weg zu einem tatsächlichen Vorteil, konnte im Rahmen dieses Projekts aber nicht bis zu einer abgeschlossenen, quantitativ berichtbaren Messung geführt werden.

Eager-Mode-Referenzmessung. Ergänzend zum torchao-Pfad liegt für alle sechs Kombinationen eine unabhängige FPS-Messung in PyTorchs eager-mode-PTQ vor, ohne backend-optimierten Kernel (Quelle: `ptq_summary.csv`, `qat_summary.csv`). Tabelle 12 zeigt, dass auch hier PTQ und QAT in jeder Kombination *langsamer* sind als FP32 (Faktor 0.10–0.39), am deutlichsten bei ResNet-50 unter QAT (0.10–0.11×). Dies ist ein drittes, methodisch unabhängiges Beispiel für denselben Befund wie §5.11 oben: Ohne einen Backend-Kernel, der quantisierte Arithmetik tatsächlich beschleunigt (torchao auf A100, `fbgemm` auf CPU), bleibt der theoretische PoT-Bitshift-Vorteil ungenutzt, unabhängig davon, mit welcher Software-Schicht gemessen wird.

5.12 Accuracy-vs-Loss-Robustheits-Check

Um zu prüfen, ob die in §5.6 berichteten Befunde artefaktisch von der Wahl der Loss-basierten

Schaden-Definition (§4.3) abhängen, wird die identische Analyse mit genauigkeitsbasiertem Schaden wiederholt (`weight_ablation_canonical_v2.csv` und dessen ImageNet100-Variante, `weight_ablation_canonical_correlation_v2.csv`). Die quantitativen Kernaussagen bleiben unverändert.

Die wahre Top-Damage-Schicht ist in allen 6 von 6 PTQ-Kombinationen identisch zwischen der Loss- und der Genauigkeits-basierten Definition (`conv1`, `conv1`, `conv1`, `conv1`, `conv1`, `conv1` für die sechs Kombinationen aus Tabelle 6). Kein einziges der 18 Score×Kombination-Korrelationsvorzeichen kehrt sich zwischen den beiden Definitionen um.

Was sich verschiebt, ist die Signifikanz-Zusammensetzung. Unter genauigkeitsbasiertem Schaden sind 3 von 18 Korrelationen bei $p < 0.05$ signifikant (`cnn`/CIFAR10 S_{raw} , ResNet-50/ImageNet100 S_{raw} , `cnn`/ImageNet100 S_{pert}), unter loss-basiertem Schaden 5 von 18 (§5.6), teils dieselben, teils andere Score×Kombination-Paare. Dass mehr Korrelationen unter dem Loss-Ziel signifikant werden, ist konsistent mit der in §4.3 genannten Motivation, den Loss als die für Gleichung (1) theoretisch passendere, kontinuierlichere Zielgröße zu verwenden.

Abbildung 6 macht diese Übereinstimmung sichtbar, indem sie für alle 18 Score×Kombination-Paare die loss-basierte gegen die genauigkeitsbasierte Spearman- ρ aufträgt. Alle Punkte liegen nahe der Diagonalen $y = x$ und, entscheidend, auf derselben Seite der Null-Linie wie ihr loss-basiertes Gegenstück, was die in §5.6 berichtete qualitative Konsistenz noch einmal direkt zeigt statt sie nur zusammenzufassen. Insgesamt stützt dieser Robustheits-Check die Kernaussage aus §5.6 unabhängig von der gewählten Schaden-Metrik, eine qualitative Bestätigung, dass Loss die richtige Zielgröße für diese Analyse ist (§4.3), ohne dass Genauigkeit dafür jemals als Alternative geführt wurde.

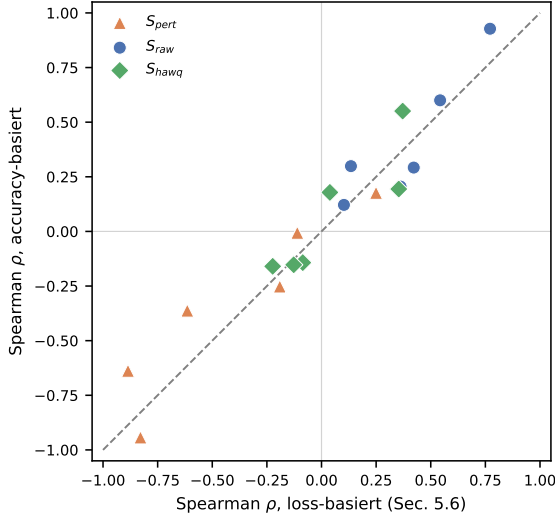


Abbildung 6: Spearman- ρ je der 18 Score×Kombination-Paare, loss-basiert (x-Achse, §5.6) gegen genauigkeitsbasiert (y-Achse, dieser Abschnitt). Graue Diagonale = $y = x$. Erzeugt von `notebooks/13_accuracy_loss_robustness.ipynb`.

6 Diskussion

6.1 Warum HAWQ-V2 unter PoT scheitert

Die Taylor-Motivation hinter dem HAWQ-V2-Produkt (Gleichung (1)) setzt implizit voraus, dass die Störungsgröße $\|\Delta W_l\|^2$ über die Schichten hinweg vergleichbar skaliert, sodass die Krümmung $\text{Tr}(H_l)$ die eigentliche Rangfolge dominiert. Unter *uniformer* Quantisierung ist diese Annahme plausibel. Die Störungsgröße hängt dort im Wesentlichen von der gewählten Bit-Breite und dem Wertebereich einer Schicht ab, und beide Größen unterscheiden sich zwischen den Schichten eines Netzes oft nur moderat.

Unter PoT bricht diese Annahme zusammen. Wie in §2.1 erläutert, hängt die PoT-Störungsgröße primär davon ab, wie gut die spezifischen Gewichtswerte einer Schicht zufällig mit dem logarithmischen PoT-Gitter übereinstimmen, was eine weitgehend von der Krümmung dieser Schicht unabhängige Eigenschaft ist. Empirisch zeigt sich das genau an der Schicht, die für die gesamte Analyse zentral ist. `conv1` hat in den meisten Kombinationen die höchste Sensitivität (höchster S_{raw} -Rang, Tabelle 6), aber eine der kleinsten Störungsgrößen, sodass die beiden Faktoren des HAWQ-V2-Produkts hier *entgegengesetztes* Verhalten zeigen. Konsistent damit korreliert S_{pert} in 5 von 6 Kombinationen *negativ* mit dem gemessenen Schaden, exakt entgegengesetzt zu S_{raw} s positiver Korrelation in denselben 5 Kombi-

nationen (§5.6). Das Produkt S_{hawq} wird dadurch systematisch nach unten gezogen. Statt dass sich Krümmungs- und Störungssignal wie in der Taylor-Motivation vorgesehen sinnvoll ergänzen, heben sie sich teilweise auf.

Wichtig ist die Einordnung dieses Befunds. Er ist kein allgemeines Scheitern Krümmungs-basierter Sensitivität, sondern ein spezifisches Scheitern der Produktbildung genau unter PoT. Das Produkt funktioniert nur dann als sinnvoller Sensitivitäts-Score, wenn $\|\Delta W\|^2$ zumindest unkorreliert mit der Krümmungs-basierten Sensitivität einer Schicht ist. PoT verletzt dies strukturell, da die Gitter-Ausrichtung eine Eigenschaft der Gewichtsverteilung und unabhängig von der Krümmung ist, und für `conv1` sind beide direkt gegenläufig (höchste Sensitivität, kleinste Störung). Dies erklärt, warum eine etablierte, breit genutzte Sensitivitätsformulierung spezifisch unter diesem Quantisierungsschema in die Irre führt.

6.2 Konfigurations-Sensitivität als methodisches Problem

Das Rekonziliations-Ledger (§5.4) ist mehr als eine Fußnote. Dass zwischen den historischen und den kanonischen Werten derselben nominellen Größe zwei Größenordnungen Abweichung liegen, ohne dass eine einzelne Konfigurationsentscheidung im gesweepen Parameterraum sie erklärt, zeigt, wie fragil ein einzelner, nicht dokumentierter Trace-Wert als Forschungsartefakt ist. Die praktische Implikation reicht über dieses Projekt hinaus. Publiizierte Krümmungs-basierte Sensitivitätsanalysen, die ihre Konfiguration (Loss-Reduktion, BN-Fusionsstatus, Proben-Budget, Datensplit, Zufalls-Seeds) nicht explizit dokumentieren, sind mit hoher Wahrscheinlichkeit schwer bis gar nicht reproduzierbar. Dies liegt nicht daran, dass die zugrundeliegende Methode fehlerhaft wäre, sondern daran, dass der Wertebereich, den unterschiedliche plausible Konfigurationsentscheidungen erzeugen können, in den eigenen Messungen mehrere Größenordnungen umfasst (`drift_diagnosis.csv`, §5.4).

Daraus lässt sich eine konkrete Empfehlung für zukünftige Arbeiten in diesem Bereich ableiten. Eine kanonische, explizit versionierte Konfigurationsdatei (wie die vorliegende `trace_config.json`) sollte Standardpraxis sein, ebenso wie das Protokollieren, welche historischen Werte mit welcher Konfiguration reproduziert werden konnten und welche nicht, statt nur den zuletzt gemessenen Wert ohne diese Historie zu berichten.

6.3 Gewichts- vs. Aktivierungsregime

Die Dekomposition aus §5.2 zeigt, dass unter PTQ in allen drei getesteten Architekturen der Gewichtsanteil den Genauigkeitsverlust auf CIFAR10 dominiert. Da die Hessian-Analyse ausschließlich Gewichtsschaden erfasst, deckt sie damit, zumindest für PTQ auf CIFAR10, den tatsächlich dominierenden Verlustmechanismus ab. Unter QAT kehrt sich das Bild um. Dort dominiert der Aktivierungsanteil, und eine Übertragung der Hessian-basierten Gewichts-Sensitivitätsanalyse auf diesen Fall wäre nicht gerechtfertigt. Diese Beobachtung, konsistent mit der grundsätzlich unterschiedlichen Natur von Gewichts- und Aktivierungsschaden aus §2.5, ist der Kern der Antwort auf F3. Die Hessian-basierte Sensitivitätsanalyse ist nicht universell auf jedes Quantisierungsregime anwendbar, sondern regimeabhängig, und ihre Anwendbarkeit sollte vor jedem Einsatz explizit geprüft werden, etwa durch eine Dekomposition wie in §5.2, statt vorausgesetzt zu werden. Da die verfügbare Dekompositionsdatei nur CIFAR10 abdeckt, bleibt offen, ob sich das Bild auf ImageNet100 wiederholt (§6.6).

6.4 PTQ vs. QAT Krümmungsstruktur

Die Ergebnisse aus §5.9 beantworten F1 mit einer auf den ersten Blick überraschenden Kombination. Einerseits verändert QAT die *relative* schichtweise Krümmungsstruktur eines Netzes kaum, die Spearman-Korrelation zwischen den PTQ- und QAT-Trace-Profilen liegt bei $\rho = 0.95\text{--}1.00$ über alle sechs Kombinationen. Andererseits bricht die Beziehung zwischen dieser Krümmung und dem tatsächlich beobachteten Schaden unter QAT fast vollständig zusammen (0 von 18 loss-basierten Korrelationen signifikant, gegenüber 5 von 18 unter PTQ, §5.6). Auf den ersten Blick widerspricht sich das, dieselbe Krümmungslandschaft sollte doch eine ähnliche Vorhersagekraft behalten. Die Auflösung liegt darin, dass sich nicht die Krümmung, sondern das *Ziel* verschiebt. Unter QAT hat sich das Netz bereits an den Quantisierungsfehler angepasst (§2.1), sodass die isolierte Quantisierung einer einzelnen Schicht den Loss oft nicht mehr verschlechtert, sondern sogar leicht verbessert (§5.2). Die Hessian-Krümmung bleibt also ein stabiles Merkmal der Netzarchitektur, ihre Interpretation als Schadens-Prädiktor setzt aber voraus, dass eine Störung tatsächlich schadet, eine Voraussetzung, die PTQ erfüllt und QAT gerade durch sein Trainingsziel systematisch unterläuft. Praktisch bedeutet dies, dass eine Hessian-basierte Sensitivitätsanalyse für QAT-Modelle nicht ohne Weiteres von

PTQ-Modellen übernommen werden sollte, selbst wenn die zugrunde liegende Krümmungslandschaft nahezu identisch aussieht.

6.5 Deployment-Realität vs. theoretischer Vorteil

Der in §2.1 beschriebene Bitshift-Vorteil von PoT ist eine Eigenschaft der Arithmetik und nicht automatisch eine Eigenschaft jeder verfügbaren Software-/Hardware-Kombination. Die Deployment-Ergebnisse (§5.11) zeigen diesen Unterschied deutlich. Auf einer Standard-Rechenzentrums-GPU (A100) mit einem aktuellen, aber in seiner Faltungs-Kernel-Abdeckung eingeschränkten Quantisierungs-Framework (torchao 0.17) bringt die INT8-Quantisierung *keinen* Laufzeitvorteil und ist in der naheliegendsten Konfiguration deutlich langsamer als FP32. Auf einer CPU mit dedizierter INT8-Hardware-Unterstützung (AVX-512 VNNI) und einem Backend, das tatsächlich quantisierte Faltungs-Kernel implementiert (fbgemm), ist der Weg zu einem echten Vorteil dagegen mechanistisch nachvollziehbar vorhanden, auch wenn er im Rahmen dieses Projekts nicht bis zu einer abgeschlossenen quantitativen Messung geführt werden konnte.

Die praktische Lehre daraus ist, dass es entscheidend von der Reife und dem Umfang der eingesetzten Inferenz-Software abhängt, ob ein theoretisch hardware-attraktives Quantisierungsschema tatsächlich einen Vorteil liefert, und nicht nur vom Schema selbst. Die Speicherreduktion (theoretisch $4\times$ für Gewichte, §5.11) bleibt der arithmetisch verlässlichere Vorteil, unabhängig davon, ob ein optimierter Rechenkernel existiert.

6.6 Limitationen

- **Ein Trainings-Seed pro Kombination.** Für keine der sechs Architektur×Datensatz-Kombinationen wurde über mehrere Trainings-Seeds gemittelt. Die Cross-Seed-Varianz der Top-Damage-Layer-Identifikation ist daher nicht charakterisiert; einzelne Schicht-Ränge könnten sich unter einem anderen Trainings-Seed verschieben. Als robust gilt das qualitative Muster, also die Top-Damage-Layer-Identität, der S_{raw} -vs- S_{hawq} -Vergleich und die top- k^* -Überlappungen, nicht die exakte Einzelrang-Stabilität.
- **Ungleiche Proben-Budgets zwischen Datensätzen** (§4.2): CIFAR10 und ImageNet100

wurden mit unterschiedlichem Hutchinson-Proben-Budget vermessen, aus rechnerischer Notwendigkeit. Ein Teil der schwächeren Korrelation auf CIFAR10 (§5.6) könnte auf Messrauschen statt auf einen echten Datensatzeffekt zurückgehen.

- **Nur drei Architekturen, nur zwei Datensätze.** Die Generalisierbarkeit auf andere CNN-Familien (z.B. EfficientNet, MobileNet) oder auf nicht-Vision-Domänen ist ungetestet.
- **Nur PoT, keine uniforme oder gemischt-präzise Quantisierung.** Die Kernaussage, dass das HAWQ-V2-Produkt spezifisch durch PoT-Grid-Alignment-Effekte irregeführt wird (§6.1), ist eine PoT-spezifische Erklärung; ob und wie stark sie sich auf uniforme oder gemischt-präzise Schemata überträgt, ist nicht getestet.
- **PTQ als primärer Fokus.** QAT-Ergebnisse (§5.9) werden berichtet, sind aber durchweg schwächer signifikant und methodisch komplizierter zu interpretieren, da isolierte QAT-Quantisierung den Loss teils verbessert statt verschlechtert.
- **KFAC-Zerlegung gescheitert** (§5.8): Die architektonische Ursachensuche über KFAC liefert keine belastbare Erklärung; die drei aufgestellten Hypothesen bleiben unentschieden.
- **ImageNet100-Auflösungskontrast für Phase 6 nicht auswertbar:** Der für eine Diskriminierung der KFAC-Hypothesen vorgesehene Kontrast zwischen den Eingabeauflösungen von CIFAR10 und ImageNet100 (`resolution_contrast_ratio`) ist in den verfügbaren Daten durchweg nicht bestimmt.
- **Unvollständige CPU-Deployment-Messung** (§5.11): Der fbgemm-CPU-Pfad wurde ausgeführt und mechanistisch verifiziert, aber kein vollständiger Durchsatz-Sweep mit finaler Zusammenfassung ist verfügbar.

6.7 Ausblick

Mehrere Anschlussrichtungen ergeben sich direkt aus den hier dokumentierten Negativresultaten und offenen Fragen. Erstens könnte eine RMT-informierte Zerlegung, die explizit zwischen Bulk- und Ausreißer-Eigenwerten des Hessian-Spektrums unterscheidet (§2.4), dort ansetzen, wo die KFAC-Faktorisierung

gescheitert ist (§5.8), möglicherweise mit einer größeren, aber numerisch stabileren Zerlegung. Zweitens sollte die zentrale Erklärung dieses Projekts (§6.1) systematisch gegen uniforme und gemischt-präzise Quantisierungsschemata getestet werden, um zu prüfen, ob das HAWQ-V2-Scheitern tatsächlich PoT-spezifisch ist oder breiter auftritt. Drittens ist eine Ausweitung auf Transformer-Architekturen naheliegend, insbesondere da die einzige vorherige direkte Validierung der HAWQ-V2-Metrik [7] bereits im LLM-Setting stattfand (§3); ein direkter CNN-vs-Transformer-Vergleich unter identischer Methodik wäre aufschlussreich. Viertens sollte eine Multi-Seed-Validierung die in §6.6 genannte Seed-Varianzlücke schließen. Der unmittelbare nächste Schritt ist jedoch bereits erfolgt, nämlich die Einreichung der Kernergebnisse zu F2 als Workshop-Paper bei AXIOM @ NeurIPS 2026 (§7), das der Fachcommunity zur Diskussion vorliegt.

7 Fazit

Dieses Projekt hat untersucht, ob Hessian-basierte Krümmungsmetriken vorhersagen können, welche Schichten eines CNN unter der Power-of-Two-Quantisierung am stärksten geschädigt werden, und liefert auf alle drei Forschungsfragen eine klare, wenn auch nuancierte Antwort.

Zu **F1** (schichtweise Krümmungsstruktur unter PTQ vs. QAT): Die Random-Init-Kontrolle (§5.3) zeigt, dass die für die Analyse zentrale `conv1`-Krümmungsspitze in allen drei Architekturen durch Training entsteht und nicht bereits bei Initialisierung vorhanden ist; die Fusions-Analyse (§5.5) zeigt weiter, dass diese Spitze überwiegend ein Effekt der Batch-Normalisierungs-Fusion ist und nicht der PoT-Quantisierung selbst, mit Ausnahme des `cnn`, wo PTQ zusätzlich moderat verstärkt. Der direkte PTQ-vs-QAT-Vergleich (§5.9) zeigt ein zunächst kontraintuitives Bild: Die relative Krümmungsstruktur selbst bleibt zwischen PTQ- und QAT-Checkpoints nahezu unverändert ($\rho = 0.95\text{--}1.00$ über alle sechs Kombinationen), während die Beziehung dieser Krümmung zum tatsächlichen Schaden unter QAT fast vollständig zusammenbricht (0 von 18 signifikant, gegenüber 5 von 18 unter PTQ). Der Grund liegt nicht in der Krümmung, sondern im Ziel. Die isolierte Schichtquantisierung verbessert unter QAT häufig den Loss gegenüber der FP32-Baseline, statt ihn zu verschlechtern (§5.2), so dass dieselbe Krümmungslandschaft ein grundlegend anderes Zielsignal vorherzusagen versucht (§6.4).

Zu **F2** (der Kernfragestellung): Kein einzelner Sco-

re ist universell zuverlässig, um die am stärksten geschädigte Schicht zu identifizieren. Die rohe Krümmung (S_{raw}) identifiziert die wahre Top-Damage-Schicht in 3 von 6 Kombinationen exakt (Rang 1) und wird in ihrer top- k^* -Überlappung von keinem der beiden anderen Scores übertroffen. Das HAWQ-V2-Produkt scheitert spezifisch daran, dass die PoT-Störungsgröße mit der wahren Sensitivität *antikorreliert*, statt neutral oder positiv zu sein (§6.1), was ein PoT-spezifischer Mechanismus und keine allgemeine Schwäche Krümmungs-basierter Sensitivität ist. Welche Schicht konkret am stärksten geschädigt wird, hängt zudem stark von Architektur und Datensatz ab (**conv1** auf ImageNet100 in allen drei Architekturen, aber uneinheitlich auf CIFAR10, Tabelle 6).

Zu **F3** (Gewichts- vs. Aktivierungsregime und Anwendbarkeit): Unter PTQ auf CIFAR10 dominiert der Gewichtsschaden in allen drei Architekturen, unter QAT dominiert der Aktivierungsschaden (§5.2). Die Hessian-basierte Sensitivitätsanalyse, die ausschließlich Gewichtsschaden erfasst, ist damit nicht universell anwendbar, sondern regimeabhängig, und ihre Anwendbarkeit sollte vor jedem Einsatz explizit geprüft werden.

Über diese drei Fragen hinaus liefert das Projekt zwei methodische Beiträge mit Reichweite über den eigenen Anwendungsfall hinaus: das Rekonziliations-Ledger als dokumentierter Beleg dafür, wie stark Trace-Messungen von unausgesprochenen Konfigurationsentscheidungen abhängen (§5.4), und die Deployment-Evaluation als Beleg dafür, dass ein theoretischer Hardware-Vorteil ohne ausgereifte Software-Unterstützung nicht automatisch real wird (§5.11). Ebenso gehört das dokumentierte Scheitern der KFAC-Zerlegung (§5.8) zu den ehrlich zu berichtenden Ergebnissen dieses Projekts.

Die Kernergebnisse zu F2, also der direkte Test der HAWQ-V2-Formulierung gegen gemessenen Schaden, das Rekonziliations-Ledger und die Metrik-Konkordanz-Analyse, wurden in kompakter Form als Workshop-Paper bei **AXIOM @ NeurIPS 2026** eingereicht. Dieser Bericht dokumentiert das Projekt in seiner vollen Breite, einschließlich der Phasen, Kontrollen und Negativresultate, die im begrenzten Format eines Workshop-Papers keinen Platz fanden, aber für ein vollständiges Bild der methodischen Arbeit dieses Projekts wesentlich sind.

Literatur

- [1] Yoshua Bengio, Nicholas Léonard und Aaron Courville. „Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation“. In: *arXiv preprint arXiv:1308.3432* (2013). arXiv: 1308.3432.
- [2] Zhen Dong u. a. „HAWQ-V2: Hessian Aware trace-Weighted Quantization of Neural Networks“. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020. arXiv: 1911.03852.
- [3] Zhen Dong u. a. „HAWQ: Hessian AWARE Quantization of Neural Networks with Mixed-Precision“. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2019. arXiv: 1905.03696.
- [4] Mahmoud Elgenedy. *Power-of-Two (PoT) Weights in Large Language Models (LLMs)*. 2025. arXiv: 2506.00315.
- [5] Roger Grosse und James Martens. „A Kronecker-factored Approximate Fisher Matrix for Convolution Layers“. In: *Proceedings of the 33rd International Conference on Machine Learning (ICML)*. 2016. arXiv: 1602.01407.
- [6] Kaiming He u. a. *Deep Residual Learning for Image Recognition*. 2015. arXiv: 1512.03385.
- [7] Joshua Hill. *Saturation Makes Quantization Error Additive: A Coverage Model with a Certificate*. 2026. arXiv: 2607.12266 [cs.LG]. URL: <https://arxiv.org/abs/2607.12266>.
- [8] Sergey Ioffe und Christian Szegedy. *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. 2015. arXiv: 1502.03167.
- [9] Ryo Karakida, Shotaro Akaho und Shun-ichi Amari. „Universal Statistics of Fisher Information in Deep Neural Networks: Mean Field Approach“. In: *Proceedings of the 22nd International Conference on Artificial Intelligence and Statistics (AISTATS)*. 2019. arXiv: 1806.01316.
- [10] Daya Khudia u. a. *FBGEMM: Enabling High-Performance Low-Precision Deep Learning Inference*. 2021. arXiv: 2101.05615.
- [11] Young Kim u. a. „Non-zero Grid for Accurate 2-Bit Additive Power-Of-Two CNN Quantization“. In: *IEEE Access* (2023).

- [12] Edward H. Lee u. a. „LogNet: Energy-efficient Neural Networks Using Logarithmic Computation“. In: *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2017, S. 5900–5904.
- [13] Xinlin Li u. a. „DenseShift: Towards Accurate and Efficient Low-Bit Power-of-Two Quantization“. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2023, S. 16964–16974.
- [14] Yuhang Li u. a. „BRECQ: Pushing the Limit of Post-Training Quantization by Block Reconstruction“. In: *International Conference on Learning Representations (ICLR)*. 2021. arXiv: 2102.05426.
- [15] Ji Lin u. a. „AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration“. In: *Proceedings of Machine Learning and Systems (MLSys)*. Best Paper Award. 2024. arXiv: 2306.00978.
- [16] James Martens und Roger Grosse. „Optimizing Neural Networks with Kronecker-factored Approximate Curvature“. In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. 2015. arXiv: 1503.05671.
- [17] Daisuke Miyashita, Edward H. Lee und Boris Murmann. „Convolutional Neural Networks using Logarithmic Data Representation“. In: *arXiv preprint arXiv:1603.01025* (2016). arXiv: 1603.01025.
- [18] Dominika Przewłocka-Rus u. a. „Power-of-Two Quantization for Low Bitwidth and Hardware Compliant Neural Networks“. In: *tinyML Research Symposium*. 2022. arXiv: 2203.05025.
- [19] Levent Sagun, Léon Bottou und Yann LeCun. „Singularity of the Hessian in Deep Learning“. In: *arXiv preprint arXiv:1611.07476* (2016). arXiv: 1611.07476.
- [20] Levent Sagun u. a. „Empirical Analysis of the Hessian of Over-Parametrized Neural Networks“. In: *arXiv preprint arXiv:1706.04454* (2017). ICLR 2018 Workshop. arXiv: 1706.04454.
- [21] Guangxuan Xiao u. a. „SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models“. In: *Proceedings of the 40th International Conference on Machine Learning (ICML)*. 2023. arXiv: 2211.10438.
- [22] Zhewei Yao u. a. „PyHessian: Neural Networks Through the Lens of the Hessian“. In: *2020 IEEE International Conference on Big Data (Big Data)*. IEEE, 2020, S. 581–590. arXiv: 1912.07145.